



验证用例报告

eHSM

OSR VR

June 30, 2026

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2026 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有 © 2014 - 2026 深圳市纽创信安科技发展有限公司保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技发展有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技发展有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

Contents

1 概述	3
2 用例描述	4
2.1 时钟和复位	4
2.1.1 scan_mode_test	4
2.1.2 sys_busrst_test	4
2.2 otp 及其映射	4
2.2.1 otp_reg_test	4
2.2.2 version_counter_test	5
2.2.3 ske_gm_en	5
2.2.4 hash_gm_en	6
2.2.5 otp_patch	6
2.3 硬件安全通路	6
2.3.1 otp_key2kmu	6
2.3.2 otp_key2kmu_1	7
2.3.3 otp_key2kmu_2	7
2.3.4 otp_key2kmu_3	7
2.3.5 otp_key2kmu_4	8
2.3.6 otp_key2kmu_5	8
2.3.7 otp_key_attribute_valid_key	9
2.3.8 otp_key_attribute	9
2.3.9 otp_key_attribute_invalid_chip_root_key	9
2.3.10 otp_key_attribute_invalid_device_root_key	10
2.3.11 otp_key_attribute_invalid_user_root_key	10
2.4 CPU <-> memory	10
2.4.1 dram_test	10
2.4.2 dram_dir_test	11
2.4.3 dram_init_test	11
2.4.4 irom_test	11
2.4.5 irom_512k_test	12
2.4.6 iram_test	12
2.4.7 iram_init_test	12
2.4.8 sbus_ram	13
2.4.9 ibus_sbus_conflict	13
2.4.10 ibus_sbus_burst_conflict	13
2.4.11 ibus_sbus_dma_conflict	14
2.5 CPU <-> 外部 bus	14
2.5.1 cpu2soc	14
2.5.2 cpu2soc_hsize	15
2.5.3 ahb_otp_bit	15
2.5.4 ahb_otp_add	16
2.5.5 ahb_cfg_bit	16
2.5.6 ahb_cfg_add	16
2.6 KMU <-> 算法 IP	17
2.6.1 seip_kmu_reg_bit	17
2.6.2 seip_kmu_reg_add	17
2.6.3 kmu_init_test	17
2.6.4 kmu_rw_test	18
2.6.5 kmu_rw_rtl_kek_test	18
2.6.6 kmu_ctrl_stick	18
2.7 CPU <-> 算法 IP <-> AHB_DMA/AXI_DMA	19

2.7.1	ske_sanity	19
2.7.2	ske_axi_ahb_dma	19
2.7.3	hfe_axi_ahb_dma	19
2.7.4	hfe_ske_serial	20
2.7.5	hfe_ske_parallel	20
2.7.6	hfe_ske_axi_parallel	20
2.7.7	ske_ahb_dma_directed	21
2.7.8	ske_axi_dma_directed	21
2.7.9	dma_corner_ske	22
2.7.10	dma_corner_hash	22
2.7.11	hfe_ske_reset	23
2.7.12	trng_sanity	23
2.7.13	pke_sanity	23
2.7.14	pke_cip_ram_test	24
2.7.15	pke_cip_sanity	24
2.7.16	pke_ram_test	24
2.8	CPU <-> SYS_CSR	25
2.8.1	sys_csr_reg_bit	25
2.8.2	sys_csr_reg_add	25
2.8.3	sys_sta0_test	25
2.8.4	sys_sta1_test	26
2.8.5	sys_direct_test	26
2.8.6	sys_clk0_test	26
2.8.7	sys_rst0_test	27
2.8.8	sys_alg_clk_test	27
2.8.9	sys_alg_rst_test	28
2.8.10	sys_axidmacfg_test	28
2.8.11	sys_int_test	28
2.9	CPU <-> MBOX <-> SOC	29
2.9.1	ahb_master_mbox_prot_test	29
2.9.2	seip_sembox_reg_bit	29
2.9.3	seip_sembox_reg_add	30
2.9.4	mbox_bf_cpurst	30
2.9.5	mbox_burst	30
2.9.6	mbox_hready_test	30
2.9.7	mbox_info_test	31
2.9.8	mbox_test	31
2.9.9	mbox_irq_test	33
2.9.10	mbox_col_int_test	34
2.10	CPU <-> UART/TIMER/WDT/CRC	36
2.10.1	uart_sanity	36
2.10.2	wdt_sanity_test	37
2.10.3	timer_sanity	37
2.10.4	seip_reg	37
2.10.5	seip_exit_irq	38
2.10.6	seip_crc_bit	38
2.11	CPU <-> EMU	38
2.11.1	emu_kmu_ram_ecc_reg	38
2.11.2	emu_kmu_ram_ecc_secp	39
2.11.3	emu_otp_key_crc_chip_root_key	39
2.11.4	emu_otp_key_crc_device_root_key	40
2.11.5	emu_otp_key_crc_user_root_key	40
2.11.6	emu_otp_key_crc_chip_key	41

2.11.7 emu_otp_key_crc_device_key	41
2.11.8 emu_otp_key_crc_illegal_key	42
2.11.9 emu_otp_key_crc_mode_random	43
2.11.10 emu_otp_key_crc_valid_key_random	43
2.11.11 dram_ecc_testmode_valid	44
2.11.12 dram_ecc_invalid	44
2.11.13 dram_ecc_force_valid	44
2.11.14 dram_ecc_byte_err_rsp	45
2.11.15 dram_ecc_err_rsp	45
2.11.16 iram_ecc_testmode_valid	46
2.11.17 iram_ecc_invalid	46
2.11.18 iram_ecc_force_valid	47
2.11.19 iram_ecc_err_rsp	48
2.11.20 iram_ecc_byte_err_rsp	48
2.11.21 irom_ecc_invalid	48
2.11.22 irom_ecc_err_rsp	49
2.11.23 irom_ecc_byte_err_rsp	49
2.11.24 pke_ram0_ecc_testmode_valid	49
2.11.25 pke_ram0_ecc_invalid	50
2.11.26 pke_ram0_ecc_force_valid	50
2.11.27 pke_ram0_sec_rst_test	51
2.11.28 pke_ram0_ded_rst_test	51
2.11.29 pke_ram1_ecc_testmode_valid	52
2.11.30 pke_ram1_ecc_invalid	52
2.11.31 pke_ram1_ecc_force_valid	53
2.11.32 pke_ram1_sec_rst_test	53
2.11.33 pke_ram1_ded_rst_test	54
2.11.34 pke_ram2_ecc_testmode_valid	54
2.11.35 pke_ram2_ecc_invalid	54
2.11.36 pke_ram2_ecc_force_valid	55
2.11.37 pke_ram2_sec_rst_test	56
2.11.38 pke_ram2_ded_rst_test	56
2.11.39 pke_ram3_ecc_testmode_valid	56
2.11.40 pke_ram3_ecc_invalid	57
2.11.41 pke_ram3_ecc_force_valid	57
2.11.42 pke_ram3_sec_rst_test	58
2.11.43 pke_ram3_ded_rst_test	58
2.11.44 axi_dma_rd_err_rsp	59
2.11.45 axi_dma_wr_err_rsp	59
2.11.46 ahb_cfg_err_rsp_rst	60
2.11.47 ahb_cfg_err_rsp_err2cpu	60
2.11.48 ahb_soc_err_rsp_rst	60
2.11.49 ahb_soc_err_rsp_err2cpu	61
2.11.50 ahb_otp_err_rsp_rst	61
2.11.51 ahb_otp_err_rsp_before_boot	62
2.11.52 ahb_otp_err_rsp_err2cpu	62
2.11.53 emu_wdt_timeout_test	62
2.11.54 emu_soc_int_test	63
2.11.55 emu_soc_int_beforeboot_test	63
2.11.56 emu_soc_int_beforeboot_rnd_test	64
2.11.57 emu_soc_int_duringboot_test	64
2.11.58 emu_soc_int_duringboot_rnd_test	64
2.11.59 emu_soc_longtime_test	65

2.11.60 emu_soc_cpurst_test	65
2.11.61 emu_soc_nbtrst_test	66
2.11.62 emu_soc_allrst_test	66
2.11.63 emu_fw_test	66
2.11.64 trng_health_test_ok_in_boot	67
2.11.65 boot_err_test	67
2.11.66 reset_before_boot_test	67
2.11.67 trng_health_test_fail_in_cpu	68
2.11.68 emu_dram_sec_rst_test	68
2.11.69 emu_dram_ded_rst_test	69
2.11.70 emu_iram_ecc_dma_test	69
2.11.71 emu_iram_sec_rst_test	69
2.11.72 emu_iram_ded_rst_test	70
2.11.73 emu_irom_sec_rst_test	70
2.11.74 emu_irom_ded_rst_test	70

1 概述

本文档描述了 OSR eHSM 的验证用例。验证用例的编写基于以下文档：

- 《OSR_eHSM_HP_Custom_Design_Technical_Manual》

基于以上文档，验证用例可以分为以下几部分：

- 时钟和复位
- otp 及其映射
- 硬件安全通路
- CPU <-> memory
- CPU <-> 外部 bus
- KMU <-> 算法 IP
- CPU <-> 算法 IP <-> AHB_DMA/AXI_DMA
- CPU <-> SYS_CSR
- CPU <-> MBOX <-> SOC
- CPU <-> UART/TIMER/WDT/CRC
- CPU <-> EMU

2 用例描述

2.1 时钟和复位

2.1.1 scan_mode_test

验证场景：

- scan mode 测试

验证步骤：

- 1) 输入随机的 scan mode, icg_en;
- 2) 随机配置 csr 的 clk ctrl 寄存器, 若 scan mode 为 1, 则时钟寄存器配置和 icg_en 无效, 检查各个模块的时钟信号;
- 3) 随机配置 csr 的 rst ctrl 寄存器, 若 scan mode 为 1, 则复位寄存器配置和 icg_en 无效, 检查各个模块的复位信号;
- 4) 重复步骤 1-3 10 次

2.1.2 sys_busrst_test

验证场景：

- trng/ske/hfe 运行过程中随机发起 hsm_with_boot 复位

验证步骤：

- 1) hfe 运行过程中, 向寄存器 SYS_RST_CTRL0 中 hsm_with_boot_rst 写有效 1bit0, 等待 cpu 解复位, 重新运行 hfe, 结果比对通过;
- 2) ske 运行过程中, 向寄存器 SYS_RST_CTRL0 中 hsm_with_boot_rst 写有效 1bit0, 等待 cpu 解复位, 重新运行 ske, 结果比对通过;
- 3) trng 运行过程中, 向寄存器 SYS_RST_CTRL0 中 hsm_with_boot_rst 写有效 1bit0, 等待 cpu 解复位, 重新运行 trng

2.2 otp 及其映射

2.2.1 otp_reg_test

验证场景：

- otp reg 测试

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE、DESTROY_MODE、UNDEFINE_MODE;
- 2) 解复位, 等待硬件 boot_done 结束, 除了 DESTROY_MODE, UNDEFINE_MODE 周期外, 其它生命周期需等待 cpu 解复位, 通过 SBUS 对 sys_csr 中的 otp 模块相关寄存器进行遍历写读测试, 判断是否符合预期;
- 3) DESTROY_MODE, UNDEFINE_MODE 周期下, 等待硬件 boot_done 结束, 检查顶层 o_hsm_status 相对应 bit 位信号是否符合预期

2.2.2 version_counter_test**验证场景:**

- version_counter 值测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, 依次对 version_counter 赋值;
- 2) 解复位, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 3) 对 sys_csr 的 otp_reg 进行遍历写读测试, 判断是否符合预期;
- 4) 复位重启

2.2.3 ske_gm_en**验证场景:**

- 禁用国密算法, ske 模块验证

验证步骤:

- 1) otp 配置国密算法使能字段为禁用或者使能, 其他字段合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 发起 ske_dma 计算, 使用安全端口 key, AHB/AXI DMA 读写通道选择随机, 输入数据长度随机, DMA 源/目的地址合法随机, 选择国密算法或非国密算法, 等待完成中断;
- 3) 检查 otp 配置国密算法使能字段为禁用, 选择国密算法时, 硬件不卡住, 结果不比较, 对应配置错误寄存器置位, otp 配置国密算法使能字段为禁用, 选择非国密算法时, 或者使能, 结果比较, 对应配置错误寄存器为 0;
- 4) 重复步骤 3-50 次

2.2.4 hash_gm_en

验证场景：

- 禁用国密算法，hash 模块验证

验证步骤：

- 1) otp 配置国密算法使能字段为禁用或者使能，其他字段合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 发起 hmac_dma 计算，使用安全端口 key，AHB/AXI DMA 读写通道选择随机，输入数据长度随机，DMA 源/目的地址合法随机，选择国密算法或非国密算法，等待完成中断；
- 3) 检查 otp 配置国密算法使能字段为禁用，选择国密算法时，硬件不卡住，结果不比较，对应配置错误寄存器置位，otp 配置国密算法使能字段为禁用，选择非国密算法时，或者使能，结果比较，对应配置错误寄存器为 0；
- 4) 重复步骤 3-50 次

2.2.5 otp_patch

验证场景：

- patch 测试

验证步骤：

- 1) otp 配置 patch 使能字段为禁用或者使能，patch 配置字段随机，其他字段合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) ibus 遍历读取 rom 空间；
- 3) 检查禁用时 patch 命中不会替换 patch_data，使能时 patch 命中；
- 4) 重复步骤 3-50 次

2.3 硬件安全通路

2.3.1 otp_key2kmu

验证场景：

- otp_key key_level 测试

验证步骤：

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；

- 2) 依次将某个 otp key 的 key_life_level 设置为非法值，其他 key 属性为合法（遍历所有 otp_key）
- 3) 解复位，等待硬件 boot_done 结束，等待 cpu 解复位；
- 4) 遍历读写 kmu 的 otp key 槽，判断读写行为是否符合预期；
- 5) 遍历使用 kmu 中的 otp key 进行 ske 计算，判断计算结果是否符合预期

2.3.2 otp_key2kmu_1

验证场景：

- otp_key key_life_cycle 测试

验证步骤：

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；
- 2) 依次将某个 otp key 的 key_life_cycle 设置为非有效，其他 key 属性为合法（遍历所有 otp_key）
- 3) 解复位，等待硬件 boot_done 结束，等待 cpu 解复位；
- 4) 遍历读写 kmu 的 otp key 槽，判断读写行为是否符合预期；
- 5) 遍历使用 kmu 中的 otp key 进行 ske 计算，判断计算结果是否符合预期

2.3.3 otp_key2kmu_2

验证场景：

- otp_key key_type 测试

验证步骤：

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；
- 2) 依次将某个 otp key 的 key_type 设置为非法值，其他 key 属性为合法（遍历所有 otp_key）
- 3) 解复位，等待硬件 boot_done 结束，等待 cpu 解复位；
- 4) 遍历读写 kmu 的 otp key 槽，判断读写行为是否符合预期；
- 5) 遍历使用 kmu 中的 otp key 进行 ske 计算，判断计算结果是否符合预期

2.3.4 otp_key2kmu_3

验证场景：

- otp_key key_life_cycle 为未烧写的测试

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) 依次将某个 otp key 的 key_life_cycle 设置为未烧写, 其他 key 属性为合法 (遍历所有 otp_key)
- 3) 解复位, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 4) 读写 kmu 的 key_life_cycle 未烧写的 otp key 槽, 判断读写行为是否符合预期;
- 5) 遍历使用 kmu 中的 otp key 进行 ske 计算, 判断计算结果是否符合预期

2.3.5 otp_key2kmu_4**验证场景:**

- otp_key key_life_cycle 为非有效、非未烧写的测试

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) 依次将某个 otp key 的 key_life_cycle 设置为非有效、非未烧写, 其他 key 属性为合法 (遍历所有 otp_key)
- 3) 解复位, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 4) 读写 kmu 的 key_life_cycle 为非有效、非未烧写的 otp key 槽, 判断读写行为是否符合预期

2.3.6 otp_key2kmu_5**验证场景:**

- otp_key key_life_cycle 为非法

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) 依次将某个 otp key 的 key_life_cycle 设置为非法, 其他 key 属性为合法 (遍历所有 otp_key)
- 3) 解复位, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 4) 遍历读写 kmu 的 otp key 槽, 判断读写行为是否符合预期;
- 5) 遍历使用 kmu 中的 otp key 进行 ske 计算, 判断计算结果是否符合预期

2.3.7 otp_key_attribute_valid_key

验证场景:

- 验证 key 属性有效, BOOT 模块和 KMU 模块 key 的初始化

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE、DSTR_MODE、NONE_MODE 生命周期, key 的属性有效, key 的生命周期为有效, 等级为 1 或 2 等级, 类型为对称非对称 key, hash 值;
- 2) 检查 boot 是否启动成功;
- 3) 启动成功后检查 key 是否解密到 kmu 中, cpu 对应存储 key 的 KMU 寄存器的写读属性;
- 4) 复位后循环以上 1-3 步骤 6 次

2.3.8 otp_key_attribute

验证场景:

- 验证 key 属性随机, BOOT 模块和 KMU 模块 key 的初始化

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE、DSTR_MODE、NONE_MODE 生命周期, key 的属性随机, key 的生命周期为随机, 等级随机, 类型随机;
- 2) 检查 boot 是否启动成功;
- 3) 启动成功后检查 key 是否解密到 kmu 中, cpu 对应存储 key 的 KMU 寄存器的写读属性;
- 4) 复位后循环以上 1-3 步骤 6 次

2.3.9 otp_key_attribute_invalid_chip_root_key

验证场景:

- 验证 otp_chip_root_key, key 生命周期为非 available, 检查该 key 初始化

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE、DSTR_MODE、NONE_MODE 生命周期, chip_root_key 的属性中 key 生命周期为非 available, 其他 key 属性随机;

- 2) 检查 boot 是否启动成功, TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、DEBUG_MODE 模式启动成功, 其它模式启动失败;
- 3) 启动成功后检查 key 是否解密到 kmu 中, cpu 对应存储 key 的 KMU 寄存器的写读属性;
- 4) 复位后循环以上 1-3 步骤 6 次

2.3.10 otp_key_attribute_invalid_device_root_key

验证场景:

- 验证 otp_chip_device_key, key 生命周期为非 available, 检查该 key 初始化

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE、DSTR_MODE、NONE_MODE 生命周期, chip_root_key 的属性中 key 生命周期为 available, device_root_key 的属性中 key 生命周期为非 available, 其他 key 属性随机;
- 2) 检查 boot 是否启动成功, TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、DEBUG_MODE、MANUFACTURE_MODE 模式启动成功, 其它模式启动失败;
- 3) 启动成功后检查 key 是否解密到 kmu 中, cpu 对应存储 key 的 KMU 寄存器的写读属性;
- 4) 复位后循环以上 1-3 步骤 6 次

2.3.11 otp_key_attribute_invalid_user_root_key

验证场景:

- 验证 BOOT 模块和 KMU 模块 key 的初始化

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE、DSTR_MODE、NONE_MODE 生命周期, chip_root_key 的属性中 key 生命周期为 available, device_root_key 的属性中 key 生命周期为 available, user_root_key 的属性中 key 生命周期为非 available, 其他 key 属性随机;
- 2) 检查 boot 是否启动成功, TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、DEBUG_MODE 模式启动成功, 其它模式启动失败;
- 3) 启动成功后检查 key 是否解密到 kmu 中, cpu 对应存储 key 的 KMU 寄存器的写读属性;
- 4) 复位后循环以上 1-3 步骤 6 次

2.4 CPU <-> memory

2.4.1 dram_test

验证场景:

- dram 读写遍历，随机测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位;
- 3) 对 dram 全闕进行随机值初始化;
- 4) 在 dram 范围内进行 burst 操作 10000 次，burst 长度随机，数据 size 随机，读写随机，起始地址随机

2.4.2 dram_dir_test

验证场景:

- dram 读写定向测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位;
- 3) 对 dram 全区域进行初始化;
- 4) 执行 10000 次写读背靠背测试，地址递增;
- 5) 进行连续 word 读两次->word 写->byte 写->word 读->word 读操作

2.4.3 dram_init_test

验证场景:

- dram 初始化测试

验证步骤:

- 1) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位;
- 2) 对 dram 全区域进行 mem 读测试，同步检查 dram_DO 信号不是 X 态或 Z 态

2.4.4 irom_test

验证场景:

- ibus 访问 irom 测试

验证步骤:

- 1) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) Ibus 对 irom 空间 single 读写数据校验;
- 3) Ibus 对 irom 空间 burst 读校验, 验证与之前写入数据是否一致

2.4.5 irom_512k_test**验证场景:**

- ibus 访问 irom 测试

验证步骤:

- 1) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) ibus 遍历读取 rom 全部可访问空间, 验证与之前后门写入数据是否一致

2.4.6 iram_test**验证场景:**

- ibus 访问 iram 测试

验证步骤:

- 1) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 使用 ibus 初始化 iram, 再使用 ibus 遍历读取 iram 全域;
- 3) 使用 ibus 初始化 iram, ibus burst 读写 ram 空间, 数据 size 随机, 数据长度随机

2.4.7 iram_init_test**验证场景:**

- iram 初始化测试

验证步骤:

- 1) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 使用 ibus 遍历读取 iram 全域

2.4.8 sbus_ram

验证场景：

- sbus 访问 iram 测试

验证步骤：

- 1) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 使用 sbus 初始化 iram，再使用 sbus 遍历读取 iram 全域；
- 3) 使用 sbus 初始化 iram，sbus burst 读写 ram 空间，数据 size 随机，数据长度随机。

2.4.9 ibus_sbus_conflict

验证场景：

- ibus 和 sbus 分别同时访问 iram、nvm 冲突测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 对 ram、nvm 进行初始化；
- 3) 并行发起 sbus 和 ibus 对 iram 的 single 随机访问；
- 4) 并行发起 sbus 和 ibus 对 nvm 的 single 随机访问

2.4.10 ibus_sbus_burst_conflict

验证场景：

- ibus 和 sbus burst 分别同时访问 iram、nvm 冲突测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 对 ram、nvm 进行初始化；
- 3) 并行发起 sbus 和 ibus 对 iram 的 burst 随机访问；
- 4) 并行发起 sbus 和 ibus 对 nvm 的 burst 随机访问

2.4.11 ibus_sbusr_dma_conflict

验证场景:

- ibus 和 sbus 和 AHB_DMA 分别同时访问 iram, ibus 和 sbus 访问 nvm 冲突测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 对 ram、nvm 进行初始化;
- 3) 并行发起 sbus 和 ibus 和 ahb_dma 对 iram 的随机访问;
- 4) 并行发起 sbus 和 ibus 对 nvm 的随机访问

2.5 CPU <-> 外部 bus

2.5.1 cpu2soc

验证场景:

- cpu 访问 soc mem 测试, 包含 remap 测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 3) 随机配置 soc 偏移地址寄存器, 其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为随机, 并发起 soc mem 读写访问, 执行 500 次;
- 4) 随机配置 soc 偏移地址寄存器, 其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为 0, 该组 soc 偏移地址高 32 为随机, 低 32 位中 31 位为随机, 其他位随机, 通过 sbus_addr[30], 选择该组, sbus_addr[29:0] 为 0, 读写访问长度随机, 执行 10 次;
- 5) 随机配置 soc 偏移地址寄存器, 其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为 0, 该组 soc 偏移地址高 32 为全 0, 低 32 位中 31 位为 0, 其他位随机, 通过 sbus_addr[30], 选择该组, sbus_addr[29:0] 为 0, 读写访问长度随机, 执行 10 次;
- 6) 随机配置 soc 偏移地址寄存器, 其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为随机, 该组 soc 偏移地址高 32 为随机, 低 32 位中 31 位为随机, 其他位随机, 通过 sbus_addr[30], 选择该组, 读写访问长度随机, sbus_addr[29:0] 最后地址达到满偏, 执行 10 次;
- 7) 随机配置 soc 偏移地址寄存器, 其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为 1, 该组 soc 偏移地址高 32 为全 1, 低 32 位中 31 位为 1, 其他位随机, 通过 sbus_addr[30], 选择该组, 读写访问长度随机, sbus_addr[29:0] 最后地址达到满偏, 执行 10 次

2.5.2 cpu2soc_hsize

验证场景：

- cpu 以不同的 hsize 大小访问 soc 测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 3) 随机配置 soc 偏移地址寄存器，其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为随机，并发起 soc mem 读写访问，执行 500 次，hsize 随机配置；
- 4) 随机配置 soc 偏移地址寄存器，其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为 0，该组 soc 偏移地址高 32 为随机，低 32 位中 31 位为随机，其他位随机，通过 sbus_addr[30]，选择该组，sbus_addr[29:0] 为 0，读写访问长度随机，执行 10 次，hsize 随机配置；
- 5) 随机配置 soc 偏移地址寄存器，其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为 0，该组 soc 偏移地址高 32 为全 0，低 32 位中 31 位为 0，其他位随机，通过 sbus_addr[30]，选择该组，sbus_addr[29:0] 为 0，读写访问长度随机，执行 10 次，hsize 随机配置；
- 6) 随机配置 soc 偏移地址寄存器，其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为随机，该组 soc 偏移地址高 32 为随机，低 32 位中 31 位为随机，其他位随机，通过 sbus_addr[30]，选择该组，读写访问长度随机，sbus_addr[29:0] 最后地址达到满偏，执行 10 次，hsize 随机配置；
- 7) 随机配置 soc 偏移地址寄存器，其中一组 soc 偏移地址寄存器低 32 位 [30] 对应期望访问的地址区间为 1，该组 soc 偏移地址高 32 为全 1，低 32 位中 31 位为 1，其他位随机，通过 sbus_addr[30]，选择该组，读写访问长度随机，sbus_addr[29:0] 最后地址达到满偏，执行 10 次，hsize 随机配置

2.5.3 ahb_otp_bit

验证场景：

- sbus 访问 otp 接口测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 3) sbus 遍历读取一定 otp 空间；
- 4) sbus burst 读取一定 otp 空间，数据 size 为字，半字，字节，数据长度随机

2.5.4 ahb_otp_add

验证场景：

- sbus 访问 otp 接口测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位;
- 3) sbus 遍历覆盖 otp 地址 toggle 空间

2.5.5 ahb_cfg_bit

验证场景：

- sbus 访问 cfg 接口测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位;
- 3) sbus 遍历读取一定 cfg 空间;
- 4) sbus burst 读取一定 cfg 空间，数据 size 为字，半字，字节，数据长度随机

2.5.6 ahb_cfg_add

验证场景：

- sbus 访问 cfg 接口测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位;
- 3) sbus 遍历覆盖 cfg 地址 toggle 空间

2.6 KMU <-> 算法 IP

2.6.1 seip_kmu_reg_bit

验证场景：

- KMU 寄存器测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 除特殊寄存器执行 reset test；
- 3) 除特殊寄存器执行 bit bash test

2.6.2 seip_kmu_reg_add

验证场景：

- 特殊 KMU 寄存器测试，KMU 里 key 可传递到外部端口测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 对寄存器 kmu_en 测试；
- 3) 对空地址进行读写测试；
- 4) kmu 可传输 key 到外部端口

2.6.3 kmu_init_test

验证场景：

- kmu 硬件初始化测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，解复位，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 通过前门依次读取 KMU 寄存器的 Key 和 Key attribute 数据，同时通过监测顶层信号 u_kbuf.DO，若此信号出现 x 态或 z 态，则报错

2.6.4 kmu_rw_test

验证场景：

- kmu 读写测试

验证步骤：

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；
- 2) 解复位，等待硬件 boot_done 结束，等待 cpu 解复位；
- 3) 依次对所有 kmu key 进行读写操作；
- 4) 遍历使用 kmu key 进行 ske 计算，判断计算结果是否符合预期

2.6.5 kmu_rw_rtl_kek_test

验证场景：

- kmu 读写测试

验证步骤：

- 1) 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；
- 2) 解复位，等待硬件 boot_done 结束，等待 cpu 解复位；
- 3) 依次对所有 kmu key 进行读写操作；
- 4) 遍历使用 kmu key 和 rtl_key_install_kek_ehsm,rtl_key_install_kek_soc 进行 ske 计算，判断计算结果是否符合预期

2.6.6 kmu_ctrl_stick

验证场景：

- kmu 寄存器测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 配置寄存器 KMU_CTRL，对 KMU_EXT 写入全 f，之后对 KMU_CTRL 进行读，比较 kmu_ctrl 配置的值与读回来的数据是否一致

2.7 CPU <-> 算法 IP <-> AHB_DMA/AXI_DMA

2.7.1 ske_sanitiy

验证场景：

- ske 通路测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 随机串行发起 ske 计算，使用安全端口 key，选择 AXI DMA 读写通道，输入数据长度随机，DMA 源/目的地址合法随机，等待完成中断，检查 ske 运算结果是否正确；
- 3) 重复步骤 2

2.7.2 ske_axi_ahb_dma

验证场景：

- ske 随机选取 dma 方式

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 3) 发起 ske_dma 计算，使用安全端口 key，AHB/AXI DMA 读写通道选择随机，加解密数据长度随机，DMA 源/目的地址合法随机，等待完成中断；
- 4) 重复步骤 3 50 次

2.7.3 hfe_axi_ahb_dma

验证场景：

- hfe 随机选取 dma 方式

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE；
- 2) otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；

- 3) 发起 hmac_dma 计算, 使用安全端口 key, AHB/AXI DMA 读写通道选择随机, 输入数据长度随机, DMA 源/目的地址合法随机, 等待完成中断;
- 4) 重复步骤 3 50 次

2.7.4 hfe_ske_serial

验证场景:

- hfe、ske 串行随机计算

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 3) 随机串行发起 ske/hmac_dma 计算, 使用安全端口 key, AHB/AXI DMA 读写通道选择随机, 输入数据长度随机, DMA 源/目的地址合法随机, 等待完成中断;
- 4) 重复步骤 3 50 次

2.7.5 hfe_ske_parallel

验证场景:

- hfe、ske 并行随机计算

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 3) 随机并行发起 ske/hmac_dma 计算, 使用安全端口 key, AHB/AXI DMA 读写通道选择随机, 输入数据长度随机, DMA 源/目的地址合法随机, 等待完成中断;
- 4) 重复步骤 3 50 次

2.7.6 hfe_ske_axi_parallel

验证场景:

- hfe、ske 并行随机 AXI_DMA 计算

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 3) 随机发起 hmac_dma 计算, 使用安全端口 key, 使用 AXI DMA, 输入数据长度随机, DMA 源/目的地址合法随机, 等待完成中断, 重复 50 次;
- 4) 随机发起 ske_dma 计算, 使用安全端口 key, 使用 AXI DMA, 输入数据长度随机, DMA 源/目的地址合法随机, 等待完成中断, 重复 150 次;
- 5) 步骤 3、4 并行执行

2.7.7 ske_ahb_dma_directed

验证场景:

- ske_ahb_dma 定向测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 发起 ske_dma bypass 模式, ahb_dma 读写, 源/目的地址非对齐/字对齐/双字对齐随机, 数据长度随机, 等待完成中断, 重复该步骤 10 次;
- 3) 发起 ske_dma bypass 模式, ahb_dma 读写, 源/目的地址非对齐/字对齐/双字对齐随机, 数据长度非对齐/字对齐/双字对齐随机, 等待完成中断, 重复该步骤 10 次;
- 4) 发起 ske_dma bypass 模式, ahb_dma 读写, 源/目的地址非对齐/字对齐/双字对齐随机, 数据长度非对齐/字对齐/双字对齐随机, 源地址与目的地址相等, 等待完成中断, 重复该步骤 5 次;
- 5) 发起 ske_dma bypass 模式, ahb_dma 读写, 源/目的地址非对齐/字对齐/双字对齐随机, 数据长度非对齐/字对齐/双字对齐随机, 目的地址小于源地址且大于源地址减读数据长度, 等待完成中断, 重复该步骤 5 次;
- 6) 发起 ske_dma bypass 模式, ahb_dma 读写, 数据长度非对齐/字对齐/双字对齐随机, 目的地址等于源地址等于 iram base addr, 等待完成中断, 重复该步骤 5 次;
- 7) 发起 ske_dma bypass 模式, ahb_dma 读写, 数据长度非对齐/字对齐/双字对齐随机, 目的地址等于源地址等于 iram 最大地址减数据长度, 等待完成中断, 重复该步骤 5 次;

2.7.8 ske_axi_dma_directed

验证场景:

- ske_axi_dma 定向测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位
- 2) 发起 ske_dma bypass 模式, axi_dma 读写, 源/目的地址非对齐/字对齐/双字对齐随机, 数据长度随机, 等待完成中断, 重复该步骤 10 次;
- 3) 发起 ske_dma bypass 模式, axi_dma 读写, 源/目的地址非对齐/字对齐/双字对齐随机, 数据长度非对齐/字对齐/双字对齐随机, 等待完成中断, 重复该步骤 10 次;
- 4) 发起 ske_dma bypass 模式, axi_dma 读写, 源/目的地址非对齐/字对齐/双字对齐随机, 数据长度非对齐/字对齐/双字对齐随机, 源地址与目的地址相等, 等待完成中断, 重复该步骤 5 次;
- 5) 发起 ske_dma bypass 模式, axi_dma 读写, 源/目的地址非对齐/字对齐/双字对齐随机, 数据长度非对齐/字对齐/双字对齐随机, 目的地址小于源地址且大于源地址减读数据长度, 等待完成中断, 重复该步骤 5 次;
- 6) 发起 ske_dma bypass 模式, axi_dma 读写, 数据长度非对齐/字对齐/双字对齐随机, 目的地址等于源地址等于 0, 等待完成中断, 重复该步骤 5 次;
- 7) 发起 ske_dma bypass 模式, axi_dma 读写, 数据长度非对齐/字对齐/双字对齐随机, 目的地址等于源地址等于 AXI 总线最大地址减数据长度, 等待完成中断, 重复该步骤 5 次;

2.7.9 dma_corner_ske

验证场景:

- ske AHB_DMA 和 AXI_DMA 随机测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 随机选择 AHB/AXI 读写, 发起 ske_decrypt 计算, 使用安全端口 key, AHB/AXI DMA 读写通道选择随机, 加解密数据长度最大, DMA 源/目的地址合法随机, 运算期间随机复位;
- 3) 重复步骤 1-2 20 次

2.7.10 dma_corner_hash

验证场景:

- hash AHB_DMA 和 AXI_DMA 随机测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 随机选择 AHB/AXI 读写, 发起 hash_dma 计算, 使用安全端口 key, AHB/AXI DMA 读写通道选择随机, 加解密数据长度最大, DMA 源/目的地址合法随机, 运算期间随机复位;
- 3) 重复步骤 1-2 20 次

2.7.11 hfe_ske_reset

验证场景:

- hfe/ske 运算过程中随机复位测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 随机发起 ske/hfe dma 计算, 运算过程中随机发起系统硬复位, 一段时间后解复位, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 3) 重复步骤 2 7 次

2.7.12 trng_sanitiy

验证场景:

- trng 通路测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 使能 trng 数据中断, 使能随机数发生器;
- 3) 发起 trng reseed, 等待中断, 通过 TRNG_DR 寄存器读取输出数据, 清中断;
- 4) 重复步骤 3 4 次

2.7.13 pke_sanitiy

验证场景:

- pke 定向测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 等待 pke ram 初始化结束;
- 3) 配置中断使能, 并进行 4096 大数模逆计算;
- 4) 等待完成中断, 计算结果符合预期;

2.7.14 pke_cip_ram_test

验证场景:

- pke ram 读写测试, 使能 cipher

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 等待 pke ram 初始化结束, 配置 pke cipher 相关寄存器为随机值并使能 cipher;
- 3) 对 pke ram 进行遍历读写操作;
- 4) 对 pke ram 发起随机 burst 访问操作, 数据 size 随机, 读写随机, SINGLE; 执行 5000 次

2.7.15 pke_cip_sanity

验证场景:

- pke 定向测试, 使能 cipher

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 等待 pke ram 初始化结束, 配置 pke cipher 相关寄存器为随机值并使能 cipher;
- 3) 配置中断使能, 并进行 4096 大数模逆计算;
- 4) 等待完成中断, 计算结果符合预期;

2.7.16 pke_ram_test

验证场景:

- pke ram 读写测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 等待 pke ram 初始化结束;
- 3) 对 pke ram 进行遍历读写操作;
- 4) 对 pke ram 发起随机 burst 访问操作, 数据 size 随机, 读写随机, SINGLE; 执行 5000 次

2.8 CPU <-> SYS_CSR

2.8.1 sys_csr_reg_bit

验证场景:

- 系统寄存器测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 除特殊寄存器执行 reset test
- 3) 除特殊寄存器执行 bit bash test

2.8.2 sys_csr_reg_add

验证场景:

- 特殊系统寄存器测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 对特殊寄存器执行 reset test
- 3) 对特殊寄存器执行读写测试

2.8.3 sys_sta0_test

验证场景:

- SYS_STA0 寄存器

验证步骤:

- 1) otp 配置 life cycle
- 2) 读取 SYS_STA0 复位值, 不同 life cycle 下读取复位值不尽相同;
- 3) 向寄存器 SYS_STA0 写入随机值;
- 4) 从 CPU 顶层 force trace_interrupt 等 7bit 信号的值为 5a/25/7f/0, 读取寄存器 STA0 的值正确;
- 5) 重复步骤 1-4, 遍历 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 6) DESTROY_MODE 和 UNDEFINE_MODE 模式下, 无法读写寄存器, 直接通过层次调用检查是否处于自毁或自定模式

2.8.4 sys_sta1_test

验证场景：

- SYS_STA1 寄存器

验证步骤：

- 1) 读取 SYS_STA1 的复位值为 0；
- 2) 向 SYS_STA1 写入 5a5a_5a5a,a5a5_a5a5_, 全 1, 全 0, 并读取正确；
- 3) 重复 n 次写入随机值, 并读取正确

2.8.5 sys_direct_test

验证场景：

- CPU/SYS 复位测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位；
- 2) 配置 SYS_RST_CTRL0 寄存器, 等待 CPU、SYS 复位；
- 3) 读取 SYS_RST_CTRL0, 校验与配置的值是否一致；

2.8.6 sys_clk0_test

验证场景：

- SYS_CLK_CTRL0/1 寄存器

验证步骤：

- 1) 读取 SYS_CLK_CTRL0 的复位值为 32'h00ff_1d00；
- 2) 写入无效的数值, 低八位非 0x5a, 读取正确；
- 3) 向寄存器 SYS_CLK_CTRL0 中第 n 位写有效 1bit0, 并读取正确, 读取 SYS_CLK_CTRL1 的值为 32'h0031_1100, 向寄存器 SYS_CLK_CTRL0 写全 1, 并读取正确；
- 4) 重复步骤 3 遍历所有非 reserved 位；
- 5) 遍历向寄存器 SYS_CLK_CTRL1 中第 n 位写有效 1bit0, 并读取正确, 读取 SYS_CLK_CTRL0 的值为 32'h00ff_1d00, 向寄存器 SYS_CLK_CTRL1 写全 1, 并读取正确；
- 6) 重复步骤 5 遍历所有非 reserved 位；

2.8.7 sys_rst0_test

验证场景:

- SYS_RST_CTRL0/1 寄存器

验证步骤:

- 1) 读取 SYS_RST_CTRL0 的复位值为 32'h00ff_1f00;
- 2) 写入无效的数值, 低八位非 0x5a, 读取正确;
- 3) 向寄存器 SYS_RST_CTRL0 中第 n 位写有效 1bit0, 并读取正确, 读取 SYS_RST_CTRL1 的值为 32'h0031_1100, 向寄存器 SYS_RST_CTRL0 写全 1, 并读取正确;
- 4) 重复步骤 3 遍历所有非 reserved 位、hsm/cpu/sys/boot;
- 5) 遍历向寄存器 SYS_RST_CTRL1 中第 n 位写有效 1bit0, 并读取正确, 读取 SYS_RST_CTRL0 的值为 32'h00ff_1f00, 向寄存器 SYS_CLK_CTRL1 写全 1, 并读取正确;
- 6) 重复步骤 5 遍历所有非 reserved 位;
- 7) 向寄存器 SYS_RST_CTRL0 中 sys_rst 写有效 1bit0, 等待 5 个 clk 自动解复位, 读取 SYS_RST_CTRL0 的值为 32'h00ff_1f00, 读取 SYS_RST_CTRL1 的值为 32'h0031_1100;
- 8) 向寄存器 SYS_RST_CTRL0 中 boot_rst 写有效 1bit0, 等待 cpu 解复位, 读取 SYS_RST_CTRL0 的值为 32'h00ff_1f00, 读取 SYS_RST_CTRL1 的值为 32'h0031_1100;
- 9) 向寄存器 SYS_RST_CTRL0 中 cpu_rst 写有效 1bit0, 等待 cpu 解复位, 读取 SYS_RST_CTRL0 的值为 32'h00ff_1f00, 读取 SYS_RST_CTRL1 的值为 32'h0031_1100;
- 10) 向寄存器 SYS_RST_CTRL0 中 hsm_rst 写有效 1bit0, 等待 cpu 解复位, 读取 SYS_RST_CTRL0 的值为 32'h00ff_1f00, 读取 SYS_RST_CTRL1 的值为 32'h0031_1100;
- 11) 向寄存器 SYS_RST_CTRL0 中 hsm_with_boot_rst 写有效 1bit0, 等待 cpu 解复位, 读取 SYS_RST_CTRL0 的值为 32'h00ff_1f00, 读取 SYS_RST_CTRL1 的值为 32'h0031_1100;

2.8.8 sys_alg_clk_test

验证场景:

- SYS_ALG_CLK_CTRL0-4 寄存器

验证步骤:

- 1) 读取 SYS_ALG_CLK_CTRL0 的复位值;
- 2) 写入随机的无效的数值, 低八位非 0x5a, 读取正确, 并检查对应算法模块的时钟;
- 3) 写入随机的有效的数值, 低八位 0x5a, 读取正确, 并检查对应算法模块的时钟;
- 4) 对于寄存器 SYS_ALG_CLK_CTRL1-4 与上同理

2.8.9 sys_alg_rst_test

验证场景:

- SYS_ALG_RST_CTRL0-4 寄存器

验证步骤:

- 1) 读取 SYS_ALG_RST_CTRL0 的复位值;
- 2) 写入随机的无效的数值, 低八位非 0x5a, 读取正确, 并检查对应算法模块的复位;
- 3) 写入随机的有效的数值, 低八位 0x5a, 读取正确, 并检查对应算法模块的复位;
- 4) 对于寄存器 SYS_ALG_RST_CTRL1-4 与上同理

2.8.10 sys_axidmacfg_test

验证场景:

- SYS_AXI_DMA_CFG0-7 寄存器

验证步骤:

- 1) 读取寄存器复位值;
- 2) 向寄存器 CFG0/1/2 写随机值, 写无效, 读出仍为复位值;
- 3) 向寄存器 CFG3/4/5/6 写随机值, 读出值为写入值;
- 4) 向寄存器 CFG7 写随机值, 写无效, 读出值为 0;

2.8.11 sys_int_test

验证场景:

- SOC_STA 相关寄存器及中断

验证步骤:

- 1) 读取寄存器 SYS_SOC_STA_INT 复位值为 0, 读取寄存器 SYS_SOC_STA_INT_EN 复位值为 0, 读取寄存器 SYS_SOC_STA 复位值为 0;
- 2) 向寄存器 SYS_SOC_STA_INT 写全 1, 无效, 读出为 0;
- 3) 向寄存器 SYS_SOC_STA 写全 1, 无效, 读出为 0;
- 4) 从 HSM 顶层给 i_soc_status 赋随机值, 读取寄存器 SYS_SOC_STA_INT 正确, 读取寄存器 SYS_SOC_STA 正确, i_soc_status 赋值 0, 读取寄存器 SYS_SOC_STA_INT 值不变, 读取寄存器 SYS_SOC_STA 值为 0; 向寄存器 SYS_SOC_STA_INT 写全 0, 无效, 读出值不变; 向寄存器 SYS_SOC_STA_INT 写入读出值, 再次读取值为 0;

- 5) 从 HSM 顶层给 i_soc_status 置全 1, 读取寄存器 SYS_SOC_STA_INT 正确, 读取寄存器 SYS_SOC_STA 正确; i_soc_status 全置 0, 读取寄存器 SYS_SOC_STA_INT 正确, 读取寄存器 SYS_SOC_STA 正确; 向寄存器 SYS_SOC_STA_INT 写全 0, 无效, 读出值不变; 向寄存器 SYS_SOC_STA_INT 写第 n bit 为 1, 读出值只有第 nbit 为 0, 其他位为 1;
- 6) 重复步骤 5 遍历 n=0-31
- 7) 从 HSM 顶层给 i_soc_status 第 nbit 置 1, 读取寄存器 SYS_SOC_STA_INT 正确, 读取寄存器 SYS_SOC_STA 正确; i_soc_status 全置 0, 读取寄存器 SYS_SOC_STA_INT 正确, 读取寄存器 SYS_SOC_STA 正确;
- 8) 遍历写寄存器 SYS_SOC_STA_INTEN 第 nbit 为 1, 当且仅当与步骤 7 对应位相同时, CPU 顶层中断信号为 1, 对中断寄存器 SYS_SOC_STA_INT 写 1 清除中断后, CPU 顶层中断信号为 0;
- 9) 重复步骤 7-8, 遍历 n=0-31

2.9 CPU <-> MBOX <-> SOC

2.9.1 ahb_master_mbox_prot_test

验证场景:

- ahb port 端口测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MOD, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 限制 ahb 为 write、传输类型为 single, 传输大小为 WORD, port 随机为 DATA_ACCESS、OPCODE_FETCH、PRIVILEGED_ACCESS、BUFFERABLE、CACHEABLE 进行测试

2.9.2 seip_sembox_reg_bit

验证场景:

- sembox 寄存器测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 除特殊寄存器执行 reset test
- 3) 除特殊寄存器执行 bit bash test

2.9.3 seip_sembox_reg_add

验证场景:

- 特殊 sembox 寄存器测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 对特殊寄存器执行 reset test
- 3) 对特殊寄存器执行读写测试

2.9.4 mbox_bf_cpurst

验证场景:

- mbox 寄存器复位测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 在 CPU 复位期间读取 mbox 所有寄存器的值, 校验值复位期间为 0

2.9.5 mbox_burst

验证场景:

- mbox burst 测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 对 S2H_INFO 进行 burst 读写测试;
- 3) 对 mbox 的保留位进行 burst 读写测试.

2.9.6 mbox_hready_test

验证场景:

- mbox hready 测试

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 对 mbox 进行 hready 测试, 比较读写数据是否一致

2.9.7 mbox_info_test

验证场景:

- HSM/SOC 对 S2H_INFO, H2S_INFO 的读写属性

验证步骤:

- 1) SOC 写 S2H_INFO 随机值, 能够写入
- 2) SOC 读 S2H_INFO 正确,
- 3) HSM 读 S2H_INFO 正确;
- 4) HSM 写 S2H_INFO 随机值, 无法写入
- 5) SOC 读 S2H_INFO 正确,
- 6) HSM 读 S2H_INFO 正确
- 7) HSM 写 H2S_INFO 随机值, 能够写入
- 8) SOC 读 H2S_INFO 正确,
- 9) HSM 读 H2S_INFO 正确;
- 10) SOC 写 H2S_INFO 随机值, 无法写入
- 11) SOC 读 H2S_INFO 正确,
- 12) HSM 读 H2S_INFO 正确

2.9.8 mbox_test

验证场景:

- SOC、HSM 对 S2H_NOTE, H2S_NOTE, S2H_INT_EN, H2S_INT_EN 的读写属性。S2H_INT, H2S_INT, S2H_HSM_INT, H2S_HSM_INT, S2H_SOC_INT, H2S_SOC_INT 中断的置位和清除。

验证步骤:

- 1) MB_S2H_NOTE SOC 全写 1,
- 2) SOC 写全 0, 无法写入

- 3) HSM 写全 0, 无法写入
- 4) HSM 写 1 清第 0bit
- 5) SOC 全写 1(中断使能关闭);
- 6) 重复步骤 1 依次遍历写 1 清 nbit(n:1-29);
- 7) MB_H2S_NOTE SOC 写全 1;
- 8) HSM 写 1 至 0bit,
- 9) SOC 写 1 清 0bit,
- 10) 重复步骤 3-4, 依次遍历 nbit(n:1-29);
- 11) S2H_SOC_INT 由 SOC 写全 1,S2H_HSM_INT 由 HSM 写全 1; (写无效)
- 12) SOC 和 HSM 先后分别写 S2H_SOC_INT 和 S2H_HSM_INT 为 5a5a_a5a5,a5a5_5a5a, 双方读取都为 0;
- 13) H2S_HSM_INT 由 HSM 写全 1,H2S_SOC_INT 由 SOC 写全 1; (写无效)
- 14) SOC 和 HSM 先后分别写 H2S_SOC_INT 和 H2S_HSM_INT 为 5a5a_a5a5,a5a5_5a5a, 双方读取都为 0;
- 15) MB_S2H_NOTE SOC 全写 1, S2H_SOC_INT 双方读取为 0, S2H_HSM_INT 双方读取为全 1,
- 16) SOC 写 S2H_HSM_INT 全 0, S2H_HSM_INT 双方读取正确,
- 17) SOC 写 S2H_HSM_INT 全 1, S2H_HSM_INT 双方读取正确,
- 18) HSM 对 S2H_HSM_INT 写 1 清 0bit, S2H_HSM_INT 双方读取正确,
- 19) HSM 写 S2H_NOTE 全 1 清 0; S2H_SOC_INT 全写清;
- 20) 重复步骤 10, 依次遍历 nbit(n:1-29) 证明 S2H_HSM_INT 写清正确性;
- 21) MB_H2S_NOTE SOC 全写 1 清 0, H2S_HSM_INT 双方读取为全 1, H2S_SOC_INT 双方读取为全 0,
- 22) SOC 写 H2S_HSM_INT 全 0, H2S_HSM_INT 双方读取正确,
- 23) SOC 写 H2S_HSM_INT 全 1, H2S_HSM_INT 双方读取正确,
- 24) HSM 对 H2S_HSM_INT 写 1 清 0bit, H2S_HSM_INT 双方读取正确,
- 25) MB_H2S_NOTE 全 1,H2S_SOC_INT 全写清;
- 26) 重复步骤 12, 依次遍历 nbit(n:1-29) 证明 H2S_HSM_INT 写清正确性;
- 27) MB_S2H_NOTE HSM 全写 1 清 0, S2H_SOC_INT 双方读取为全 1, S2H_HSM_INT 双方读取为全 0,
- 28) HSM 写 S2H_SOC_INT 全 0, S2H_SOC_INT 双方读取正确,
- 29) HSM 写 S2H_SOC_INT 全 1, S2H_SOC_INT 双方读取正确,
- 30) SOC 对 S2H_SOC_INT 写 1 清 0bit, S2H_SOC_INT 双方读取正确,
- 31) MB_S2H_NOTE 全 1,S2H_HSM_INT 全写清;
- 32) 重复步骤 14, 依次遍历 nbit(n:1-29) 证明 S2H_SOC_INT 写清正确性;
- 33) MB_H2S_NOTE HSM 全写 1, H2S_HSM_INT 双方读取为 0, H2S_SOC_INT 双方读取为全 1,
- 34) HSM 写 H2S_SOC_INT 全 0, H2S_SOC_INT 双方读取正确,
- 35) HSM 写 H2S_SOC_INT 全 1, H2S_SOC_INT 双方读取正确,
- 36) SOC 对 H2S_SOC_INT 写 1 清 0bit, H2S_SOC_INT 双方读取正确,
- 37) SOC 写 H2S_NOTE 全 1 清 0; H2S_HSM_INT 全写清;

- 38) 重复步骤 16, 依次遍历 nbit(n:1-29) 证明 H2S_SOC_INT 写清正确性;
- 39) SOC 写 S2H_INT_EN 随机值, 能够写入
- 40) SOC 读 S2H_INT_EN 正确,
- 41) HSM 读 S2H_INT_EN 正确;
- 42) HSM 写 S2H_INT_EN 随机值, 无法写入
- 43) SOC 读 S2H_INT_EN 正确,
- 44) HSM 读 S2H_INT_EN 正确
- 45) HSM 写 H2S_INT_EN 随机值, 能够写入
- 46) SOC 读 H2S_INT_EN 正确,
- 47) HSM 读 H2S_INT_EN 正确;
- 48) SOC 写 H2S_INT_EN 随机值, 无法写入
- 49) SOC 读 H2S_INT_EN 正确,
- 50) HSM 读 H2S_INT_EN 正确
- 51) MB_S2H_NOTE SOC 写 1 至 0bit, S2H_HSM_INT 双方读取正确, S2H_SOC_INT 双方读取为 0,
- 52) HSM 对 S2H_HSM_INT 写 1 清 0bit, 双方读取正确;
- 53) 重复步骤 22, 依次遍历 nbit(n:1-29) 证明 S2H_HSM_INT 正确性; (此时 MB_S2H_NOTE 为全 1);
- 54) MB_H2S_NOTE HSM 写 1 至 0bit, H2S_SOC_INT 清 0,
- 55) MB_H2S_NOTE SOC 写 1 清 0bit, H2S_HSM_INT 双方读取正确, H2S_SOC_INT 双方读取为 0
- 56) HSM 对 H2S_HSM_INT 写 1 清 0bit, 双方读取正确;
- 57) 重复步骤 24, 依次遍历 nbit(n:1-29) 证明 H2S_HSM_INT 正确性; (此时 MB_H2S_NOTE 为全 0)
- 58) MB_S2H_NOTE SOC 写 1 至 0bit, S2H_HSM_INT 清 0,
- 59) MB_S2H_NOTE HSM 写 1 清 0bit, S2H_SOC_INT 双方读取正确, S2H_HSM_INT 双方读取为 0
- 60) SOC 对 S2H_SOC_INT 写 1 清 0bit, 双方读取正确;
- 61) 重复步骤 26, 依次遍历 nbit(n:1-29) 证明 S2H_SOC_INT 正确性; (此时 MB_S2H_NOTE 为全 0)
- 62) MB_H2S_NOTE HSM 写 1 至 0bit, H2S_SOC_INT 双方读取正确, H2S_HSM_INT 双方读取为 0,
- 63) SOC 对 H2S_SOC_INT 写 1 清 0bit, 双方读取正确;
- 64) 重复步骤 28, 依次遍历 nbit(n:1-29) 证明 H2S_SOC_INT 正确性; (此时 MB_H2S_NOTE 为全 1);

2.9.9 mbox_irq_test

验证场景:

- S2H_SOC_INT, H2S_SOC_INT 中断和 S2H_HSM_INT 中断至顶层中断信号 irq

验证步骤:

- 1) cpu 顶层看到中断为 0, seip 顶层无中断,

- 2) MB_S2H_NOTE 0bit 置 1, 触发 S2H_HSM_INT 0bit,
- 3) 依次遍历 0-29 使能比特, 仅对应中断使能可以使 cpu 顶层看到中断为 1, seip 顶层无中断,
- 4) 写清该中断, cpu 顶层看到中断为 0, seip 顶层无中断,
- 5) MB_S2H_NOTE 0bit 写 1 清, 触发 S2H_SOC_INT 0bit,
- 6) 依次遍历 0-29 使能比特, 仅对应中断使能可以使 seip 顶层看到中断为 1, soc 顶层无中断
- 7) 写清该中断, seip 顶层看到中断为 0, soc 顶层无中断,
- 8) 遍历步骤 1;
- 9) cpu 顶层看到中断为 0, seip 顶层无中断,
- 10) MB_H2S_NOTE 0bit 置 1, 触发 H2S_SOC_INT 0bit,
- 11) 依次遍历 0-29 使能比特, 仅对应中断使能可以使 seip 顶层看到中断为 1, cpu 顶层无中断,
- 12) 写清该中断, seip 顶层看到中断为 0, cpu 顶层无中断,
- 13) MB_H2S_NOTE 0bit 写 1 清, 触发 H2S_HSM_INT 0bit,
- 14) 依次遍历 0-29 使能比特, 仅对应中断使能可以使 cpu 顶层看到中断为 1, seip 顶层无中断
- 15) 写清该中断, cpu 顶层看到中断为 0, seip 顶层无中断,
- 16) 遍历步骤 3;

2.9.10 mbox_col_int_test

验证场景:

- S2H_NOTE, 单比特为 1 时, 对其同时写; H2S_NOTE, 单比特为 0 时, 对其同时写; HSM, SOC 同时对不同的两位 S2H_NOTE 写 1 时不置 1; HSM, SOC 同时对大于两位 S2H_NOTE 写 1 时置 1; 同时写 1 导致的 cpu 顶层中断和 SEIP 顶层中断信号。(步骤 1-26) H2S_NOTE, 单比特为 1 时, 对其同时写; H2S_NOTE, 单比特为 0 时, 对其同时写; HSM, SOC 同时对不同的两位 H2S_NOTE 写 1 时不置 1; HSM, SOC 同时对大于两位 H2S_NOTE 写 1 时置 1; 同时写 1 导致的 cpu 顶层中断和 SEIP 顶层中断信号 (步骤 27-52)

验证步骤:

- 1) HSM 对 S2H_NOTE 写全 1 清零; 清除所有中断;
- 2) SOC 对 S2H_NOTE 的第 n bit 写 1 置 1; (S2H_NOTE 只有第 n 位为 1, 其他位为 0)
- 3) SOC, HSM 同时对 S2H_NOTE 的所有 bit 写 0;
- 4) 此时, S2H_SOC_COL_INT, S2H_HSM_COL_INT 读出为 0;
- 5) SOC, HSM 同时对 S2H_NOTE 的第 n bit 写 1; (读出 S2H_NOTE nbit 为 1)
- 6) 此时, S2H_SOC_COL_INT, S2H_HSM_COL_INT 读出为 1;
- 7) SOC 写 S2H_SOC_COL_INT31bit 为 1 清该中断, 但同寄存器其他中断不影响
- 8) HSM 写 S2H_HSM_COL_INT31bit 为 1 清该中断, 但同寄存器其他中断不影响;
- 9) 重复步骤 1-6, 遍历 n 从 0-29。

- 10) HSM 对 S2H_NOTE 写全 1 清零; 清除所有中断;
- 11) SOC 对 S2H_NOTE 的所有 bit 写 1 置 1;
- 12) HSM 对 S2H_NOTE 的第 n bit 写 1 置 0; (S2H_NOTE 只有第 n 位为 0, 其他位为 1)
- 13) SOC,HSM 同时对 S2H_NOTE 的所有 bit 写 0;
- 14) 此时, S2H_SOC_COL_INT, S2H_HSM_COL_INT 读出为 0;
- 15) SOC,HSM 同时对 S2H_NOTE 的第 n bit 写 1; (读出 S2H_NOTE 为全 1)
- 16) 此时, S2H_SOC_COL_INT, S2H_HSM_COL_INT 读出为 1;
- 17) SOC 写 S2H_SOC_COL_INT31bit 为 1 清该中断, 但同寄存器其他中断不影响
- 18) HSM 写 S2H_HSM_COL_INT31bit 为 1 清该中断, 但同寄存器其他中断不影响;
- 19) 重复步骤 7-13, 遍历 n 从 0-29。
- 20) HSM 对 S2H_NOTE 写全 1 清零; 清除所有中断;
- 21) SOC,HSM 同时对 S2H_NOTE 的随机不同的某一 bit 写 1;
- 22) 此时, S2H_SOC_COL_INT, S2H_HSM_COL_INT 读出为 0;
- 23) HSM 对 S2H_NOTE 写全 1 清零; 清除所有中断;
- 24) SOC,HSM 同时对 S2H_NOTE 的写相同的随机值; (至少两位为 1)
- 25) 此时, S2H_SOC_COL_INT, S2H_HSM_COL_INT 读出为 1;
- 26) cpu 顶层看到中断为 0, seip 顶层无中断,
- 27) 触发 S2H_HSM_COL_INT (S2H_HSM_INT 中仅 31bit 为 1, 清除其他位为 0)
- 28) 依次遍历 0-29,31 使能比特, 仅对应中断使能可以使 cpu 顶层看到中断为 1,seip 顶层无中断,
- 29) HSM 写 S2H_HSM_INT 全 0 无法清该中断, SOC 写 S2H_HSM_INT 全 0+ 全 1 无法清该中断;
- 30) HSM 写 1 清该中断, cpu 顶层看到中断为 0, seip 顶层无中断,
- 31) 触发 S2H_SOC_COL_INT (S2H_SOC_INT 中仅 31bit 为 1, 清除其他位为 0)
- 32) 依次遍历 0-29,31 使能比特, 仅对应中断使能可以使 seip 顶层看到中断为 1,cpu 顶层无中断,
- 33) SOC 写 S2H_SOC_INT 全 0 无法清该中断, HSM 写 S2H_SOC_INT 全 0+ 全 1 无法清该中断;
- 34) SOC 写 1 清该中断, seip 顶层看到中断为 0, cpu 顶层无中断,
- 35) SOC 对 H2S_NOTE 写全 1 清零; 清除所有中断;
- 36) HSM 对 H2S_NOTE 的第 n bit 写 1 置 1; (H2S_NOTE 只有第 n 位为 1, 其他位为 0)
- 37) HSM,SOC 同时对 H2S_NOTE 的所有 bit 写 0;
- 38) 此时, H2S_HSM_COL_INT, H2S_SOC_COL_INT 读出为 0;
- 39) HSM,SOC 同时对 H2S_NOTE 的第 n bit 写 1; (读出 H2S_NOTE nbit 为 1)
- 40) 此时, H2S_HSM_COL_INT, H2S_SOC_COL_INT 读出为 1;
- 41) HSM 写 H2S_HSM_COL_INT31bit 为 1 清该中断, 但同寄存器其他中断不影响
- 42) SOC 写 H2S_SOC_COL_INT31bit 为 1 清该中断, 但同寄存器其他中断不影响;
- 43) 重复步骤 27-32, 遍历 n 从 0-29
- 44) SOC 对 H2S_NOTE 写全 1 清零; 清除所有中断;

- 45) HSM 对 H2S_NOTE 的所有 bit 写 1 置 1;
- 46) SOC 对 H2S_NOTE 的第 n bit 写 1 置 0; (H2S_NOTE 只有第 n 位为 0, 其他位为 1)
- 47) HSM,SOC 同时对 H2S_NOTE 的所有 bit 写 0;
- 48) 此时, HSM_COL_INT, SOC_COL_INT 读出为 0;
- 49) HSM,SOC 同时对 H2S_NOTE 的第 n bit 写 1; (读出 H2S_NOTE 为全 1)
- 50) 此时, HSM_COL_INT, SOC_COL_INT 读出为 1;
- 51) HSM 写 HSM_COL_INT31bit 为 1 清该中断, 但同寄存器其他中断不影响
- 52) SOC 写 SOC_COL_INT31bit 为 1 清该中断, 但同寄存器其他中断不影响;
- 53) 重复步骤 33-39, 遍历 n 从 0-29
- 54) SOC 对 H2S_NOTE 写全 1 清零; 清除所有中断;
- 55) SOC,HSM 同时对 H2S_NOTE 的随机不同的某一 bit 写 1;
- 56) 此时, H2S_SOC_COL_INT, H2S_HSM_COL_INT 读出为 0;
- 57) SOC 对 H2S_NOTE 写全 1 清零; 清除所有中断;
- 58) SOC,HSM 同时对 H2S_NOTE 的写相同的随机值; (至少两位为 1)
- 59) 此时, H2S_SOC_COL_INT, H2S_HSM_COL_INT 读出为 1;
- 60) cpu 顶层看到中断为 0, seip 顶层无中断,
- 61) 触发 H2S_HSM_COL_INT (H2S_HSM_INT 中仅 31bit 为 1, 清除其他位为 0)
- 62) 依次遍历 0-29,31 使能比特, 仅对应中断使能可以使 cpu 顶层看到中断为 1,seip 顶层无中断,
- 63) HSM 写 H2S_HSM_INT 全 0 无法清该中断, SOC 写 H2S_HSM_INT 全 0+ 全 1 无法清该中断;
- 64) HSM 写 1 清该中断, cpu 顶层看到中断为 0, seip 顶层无中断,
- 65) 触发 H2S_SOC_COL_INT (H2S_SOC_INT 中仅 31bit 为 1, 清除其他位为 0)
- 66) 依次遍历 0-29,31 使能比特, 仅对应中断使能可以使 seip 顶层看到中断为 1,cpu 顶层无中断,
- 67) SOC 写 H2S_SOC_INT 全 0 无法清该中断, HSM 写 H2S_SOC_INT 全 0+ 全 1 无法清该中断;
- 68) SOC 写 1 清该中断, seip 顶层看到中断为 0, cpu 顶层无中断

2.10 CPU <-> UART/TIMER/WDT/CRC

2.10.1 uart_sanity

验证场景:

- uart 通路测试

验证步骤:

- 1) user_mode 下, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 配置 uart 波特率, 发送、接受中断使能
- 3) 下发一组数据, 等待 uart TX 中断, 清除 TX 中断;
- 4) 接收一组数据, 等待 uart RX 中断, 清除 RX 中断;

2.10.2 wdt_sanity_test

验证场景：

- wdt 通路测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 配置 wdt1 初始值，中断使能
- 3) 等待 cpu 顶层 wdt 中断；
- 4) 清除 wdt 中断。

2.10.3 timer_sanity

验证场景：

- timer 通路测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 配置 timer 0 分频，初始值，使能中断并启动，等待 timer 0 中断；
- 3) 配置 timer 1 分频，初始值，使能中断并启动，等待 timer 1 中断；

2.10.4 seip_reg

验证场景：

- eHSM 内部区域 sbus 读写测试

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 使用 sbus 对 eHSM 内部区域进行寄存器检查
- 3) 使用 sbus 对 eHSM 内部 rsv 区域进行读写测试，读写属性为只读，且读取值为 0

2.10.5 seip_exit_irq

验证场景:

- 顶层信号 i_exit_irq 信号通路测试

验证步骤:

- 1) user_mode 下, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 32bit 的 i_exit_irq 信号, 每个 bit 位置 1, 等待 4 个时钟周期, 检查内部 cpu 高 32bit 中断 bit 位一致;
- 3) 32bit 的 i_exit_irq 信号, 随机置 1, 等待 4 个时钟周期, 检查内部 cpu 高 32bit 中断 bit 位一致;

2.10.6 seip_crc_bit

验证场景:

- crc 寄存器测试

验证步骤:

- 1) otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位;
- 2) 检查 CRC_ALG_CFG 复位值, 写随机值, 检查读写属性;
- 3) 检查 CRC_IN_OUT_DATA 一定空间的复位值, 写随机值, 检查读写属性

2.11 CPU <-> EMU

2.11.1 emu_kmu_ram_ecc_reg

验证场景:

- 验证 KMU_RAM_ECC 模块和 EMU 模块的对应的错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE, 以及 RAM 是否加扰; 随机选择 KMU_RAM 的位置 (非核心 key), 通过写寄存器存入数据;
- 2) 随机 ECC 使能, 随机是否注入错误, 随机使能测试模式;
- 3) 通过读寄存器读取数据;
- 4) 检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号;
- 5) 复位后循环以上 1-4 步骤 3 次;

2.11.2 emu_kmu_ram_ecc_secp

验证场景：

- 验证 KMU_RAM_ECC 模块和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，以及 RAM 是否加扰；随机选择 KMU_RAM 的位置（非核心 key），通过写寄存器存入数据；
- 2) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 3) 随机 ECC 使能，随机是否注入错误，随机使能测试模式；
- 4) 通过 kmu 安全端口读取数据；
- 5) 检查对应的 EMU 硬件错误中断寄存器，ECC 错误地址以及顶层硬件错误信号；
- 6) 检查复位寄存器随机使能对应的复位处理是否实现；
- 7) 复位后循环以上 1-6 步骤 3 次

2.11.3 emu_otp_key_crc_chip_root_key

验证场景：

- 验证 chip_root_key 的 key_crc 和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，chip_root_key 的属性中 key 生命周期为 available，其他 key 属性随机，以及 CRC 检查开关；
- 2) chip_root_key 注入 crc 错误，其他核心 key 随机注入 crc 值错误；
- 3) 检查 boot 启动过程中的 crc 电平状态，以及是否启动成功，TEST_MODE，DEVELOP0_MODE、DEVELOP_MODE，MANUFACTURE_MODE 启动成功，user_mode 如果 crc 检查开关打开会因为 chip_root_key 的 crc 错误导致启动失败，其他 boot 生命周期会失败；
- 4) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 5) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 6) 检查复位寄存器随机使能对应的复位处理是否实现；
- 7) 启动成功后对应 CRC 错误的 key 是否写入；
- 8) 复位后循环以上 1-7 步骤 3 次

2.11.4 emu_otp_key_crc_device_root_key

验证场景：

- 验证 device_root_key 的 key_crc 和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，chip_root_key，device_root_key 的属性中 key 生命周期为 available，其他 key 属性随机，以及 CRC 检查开关随机；
- 2) chip_root_key 不注入 crc 错误，device_root_key 注入 crc 错误，其他核心 key 随机注入 crc 值错误；
- 3) 检查 boot 启动过程中的 crc 电平状态，以及是否启动成功，TEST_MODE，DEBUG_MODE，DEVELOP0_MODE，DEVELOP_MODE，MANUFACTURE_MODE 启动成功，USER_MODE 如果 crc 检查开关打开会因为 device_root_key 的 crc 错误导致启动失败，MANUFACTURE_MODE 不会受 device_root_key 的 crc 错误影响，如果 crc 检查开关打开，chip_key 存在 crc 错误会导致启动失败，其他 boot 生命周期会失败；
- 4) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 5) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 6) 检查复位寄存器随机使能对应的复位处理是否实现；
- 7) 启动成功后对应 CRC 错误的 key 是否写入；
- 8) 复位后循环以上 1-7 步骤 3 次

2.11.5 emu_otp_key_crc_user_root_key

验证场景：

- 验证 OTP_KEY_CRC 模块和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，chip_root_key，device_root_key，user_root_key 的属性中 key 生命周期为 available，其他 key 属性随机，以及 CRC 检查开关随机；
- 2) chip_root_key，device_root_key 不注入 crc 错误，user_root_key 注入 crc 错误，其他核心 key 随机注入 crc 值错误；
- 3) 检查 boot 启动过程中的 crc 电平状态，以及是否启动成功，TEST_MODE，DEBUG_MODE，DEVELOP0_MODE，DEVELOP_MODE，MANUFACTURE_MODE 启动成功，USER_MODE 如果 crc 检查开关打开会因为 user_root_key 的 crc 错误导致启动失败，MANUFACTURE_MODE 不会受 user_root_key 的 crc 错误影响，如果 crc 检查开关打开，chip_key 存在 crc 错误会导致启动失败，其他 boot 生命周期会失败；

- 4) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 5) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 6) 检查复位寄存器随机使能对应的复位处理是否实现；
- 7) 启动成功后对应 CRC 错误的 key 是否写入；
- 8) 复位后循环以上 1-7 步骤 3 次

2.11.6 emu_otp_key_crc_chip_key

验证场景：

- 验证 OTP_KEY_CRC 模块和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，chip_root_key，device_root_key，user_root_key 的属性中 key 生命周期为 available，随机一个非 root_key 的 key 生命周期为 available，key 等级为 level1 (chip_key)，其他 key 属性随机，以及 CRC 检查开关随机；
- 2) chip_root_key，device_root_key，user_root_key 不注入 crc 错误，该随机的 root_key 注入 crc 错误，该随机的非 root_key 其后序号的其他核心 key 随机注入 crc 值错误，其前不注入 crc 错误；
- 3) 检查 boot 启动过程中的 crc 电平状态，以及是否启动成功，TEST_MODE，DEBUG_MODE，DEVELOP0_MODE，DEVELOP_MODE，MANUFACTURE_MODE 启动成功，USER_MODE，MANUFACTURE_MODE 如果 crc 检查开关打开会因为 chip_key 的 crc 错误导致启动失败，其他 boot 生命周期会失败；
- 4) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 5) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 6) 检查复位寄存器随机使能对应的复位处理是否实现；
- 7) 启动成功后对应 CRC 错误的 key 是否写入；
- 8) 复位后循环以上 1-7 步骤 3 次

2.11.7 emu_otp_key_crc_device_key

验证场景：

- 验证 OTP_KEY_CRC 模块和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，chip_root_key，device_root_key，user_root_key 的属性中 key 生命周期为

available, 随机一个非 root_key 的 key 生命周期为 available, key 等级为 level2 (device_key), 其他 key 属性随机, 以及 CRC 检查开关随机;

- 2) chip_root_key, device_root_key, user_root_key 不注入 crc 错误, 该随机的 root_key 注入 crc 错误, 该随机的非 root_key 其后序号的其他核心 key 随机注入 crc 值错误, 其前不注入 crc 错误;
- 3) 检查 boot 启动过程中的 crc 电平状态, 以及是否启动成功, TEST_MODE, DEBUG_MOD, DEVELOP0_MODE, DEVELOP_MODE, MANUFACTURE_MODE 启动成功, USER_MODE 如果 crc 检查开关打开会因为 device_key 的 crc 错误导致启动失败, MANUFACTURE_MODE 不会受 device_key 的 crc 错误影响, 如果 crc 检查开关打开, chip_key 存在 crc 错误会导致启动失败, 其他 boot 生命周期会失败;
- 4) 三种错误处理的复位寄存器随机使能: 复位 eHSM 寄存器, 复位除了 boot 相关模块寄存器, 复位 CPU 寄存器;
- 5) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号;
- 6) 检查复位寄存器随机使能对应的复位处理是否实现;
- 7) 启动成功后对应 CRC 错误的 key 是否写入;
- 8) 复位后循环以上 1-7 步骤 3 次

2.11.8 emu_otp_key_crc_illegal_key

验证场景:

- 验证 OTP_KEY_CRC 模块和 EMU 模块的对应的错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, chip_root_key, device_root_key, user_root_key 的属性中 key 生命周期为 available, 随机一个非 root_key 的 key 生命周期为 available, key 等级为 rsv (reserved_key), 其他 key 属性随机, 以及 CRC 检查开关随机;
- 2) chip_root_key, device_root_key, user_root_key 不注入 crc 错误, 该随机的 root_key 注入 crc 错误, 该随机的非 root_key 其后序号的其他核心 key 随机注入 crc 值错误, 其前不注入 crc 错误;
- 3) 检查 boot 启动过程中的 crc 电平状态, 以及是否启动成功, TEST_MODE, DEBUG_MOD, DEVELOP0_MODE, DEVELOP_MODE, MANUFACTURE_MODE 启动成功, USER_MODE 如果 crc 检查开关打开会因为 reserved_key 的 crc 错误导致启动失败, MANUFACTURE_MODE 不会受 reserved_key 的 crc 错误影响, 如果 crc 检查开关打开, chip_key 存在 crc 错误会导致启动失败, 其他 boot 生命周期会失败;
- 4) 三种错误处理的复位寄存器随机使能: 复位 eHSM 寄存器, 复位除了 boot 相关模块寄存器, 复位 CPU 寄存器;
- 5) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号;
- 6) 检查复位寄存器随机使能对应的复位处理是否实现;
- 7) 启动成功后对应 CRC 错误的 key 是否写入;
- 8) 复位后循环以上 1-7 步骤 3 次

2.11.9 emu_otp_key_crc_mode_random

验证场景：

- 验证 OTP_KEY_CRC 模块和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，key 属性随机，以及 CRC 检查开关；
- 2) 核心 key 随机注入 crc 值错误；
- 3) 检查 boot 启动过程中的 crc 电平状态，以及是否启动成功；
- 4) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 5) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 6) 检查复位寄存器随机使能对应的复位处理是否实现；
- 7) 启动成功后对应 CRC 错误的 key 是否写入；
- 8) 复位后循环以上 1-7 步骤 3 次

2.11.10 emu_otp_key_crc_valid_key_random

验证场景：

- 验证 OTP_KEY_CRC 模块和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，key 属性有效，以及 CRC 检查开关；
- 2) 核心 key 随机注入 crc 值错误；
- 3) 检查 boot 启动过程中的 crc 电平状态，以及是否启动成功；
- 4) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 5) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 6) 检查复位寄存器随机使能对应的复位处理是否实现；
- 7) 启动成功后对应 CRC 错误的 key 是否写入；
- 8) 复位后循环以上 1-7 步骤 3 次

2.11.11 dram_ecc_testmode_valid

验证场景:

- dram 测试模式下, 通过寄存器配置校验码有效

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为非测试模式, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 且不产生 DRAM 1bit 或多 bit 错误, 读出 DRAM 的 error 地址为 0;
- 2) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为测试模式和正确的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 且不产生 DRAM 1bit 或多 bit 错误, 读出 DRAM 的 error 地址为 0;
- 3) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为测试模式和 1bit 错误的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 产生 DRAM 1bit 错误, 不产生多 bit 错误, 读出 DRAM 的 error 地址为该随机地址;
- 4) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为测试模式和 2bit 错误的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 产生 DRAM 多 bit 错误, 不产生 1bit 错误, 读出 DRAM 的 error 地址为该随机地址;
- 5) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为测试模式和 3bit 错误的校验码, 对 DRAM 随机的地址写入随机数据, 产生 DRAM 多 bit 错误或 1bit 错误其一, 读出 DRAM 的 error 地址为该随机地址

2.11.12 dram_ecc_invalid

验证场景:

- dram 非测试模式下, 通过寄存器配置校验码无效

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为非测试模式和正确的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 且不产生 DRAM 1bit 或多 bit 错误, 读出 DRAM 的 error 地址为 0;
- 2) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为非测试模式和 1bit 错误的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 且不产生 DRAM 1bit 或多 bit 错误, 读出 DRAM 的 error 地址为 0;
- 3) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为非测试模式和 2bit 错误的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 且不产生 DRAM 1bit 或多 bit 错误, 读出 DRAM 的 error 地址为 0;
- 4) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为非测试模式和 3bit 错误的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 且不产生 DRAM 1bit 或多 bit 错误, 读出 DRAM 的 error 地址为 0

2.11.13 dram_ecc_force_valid

验证场景:

- dram 非测试模式下, 通过 force 内部信号注入数据、校验码、地址错误

验证步骤:

- 1) 对 DRAM 随机的地址写入随机数据, force i_ram_rdata 为正确的数据和校验码, 读 DRAM 为正确数据, 且不产生 DRAM 1bit 或多 bit 错误, 读出 DRAM 的 error 地址为 0;
- 2) 对 DRAM 随机的地址写入随机数据, force i_ram_rdata 为 (正确的数据和校验码) 任 1bit 取反, 读 DRAM 为正确数据, 产生 DRAM 1bit 错误, 不产生多 bit 错误, 读出 DRAM 的 error 地址为 0;
- 3) 对 DRAM 随机的地址写入随机数据, force i_ram_rdata 为 (正确的数据和校验码) 任 2bit 取反, 读 DRAM, 产生 DRAM 多 bit 错误, 不产生 1bit 错误, 读出 DRAM 的 error 地址为该随机地址;
- 4) 对 DRAM 随机的地址写入随机数据, force i_ram_rdata 为 (正确的数据和校验码) 任 3bit 取反, 读 DRAM, 产生 DRAM 多 bit 错误或 1bit 错误其一, 读出 DRAM 的 error 地址为该随机地址;
- 5) 对 DRAM 随机的地址写入随机数据, force i_ram_rdata 为正确的数据和校验码, force o_ram_addr 任 1bit 取反, 读 DRAM 为正确数据, 产生 DRAM 多 bit 错误, 不产生 1bit 错误, 读出 DRAM 的 error 地址为注入错误后的地址;
- 6) 对 DRAM 随机的地址写入随机数据, force i_ram_rdata 为正确的数据和校验码, force o_ram_addr 任 2bit 取反, 读 DRAM 为正确数据, 产生 DRAM 多 bit 错误, 不产生 1bit 错误, 读出 DRAM 的 error 地址为注入错误后的地址;
- 7) 对 DRAM 随机的地址写入随机数据, force i_ram_rdata 为正确的数据和校验码, force o_ram_addr 任 3bit 取反, 读 DRAM 为正确数据, 产生 DRAM 多 bit 错误或 1bit 错误其一, 读出 DRAM 的 error 地址为注入错误后的地址;
- 8) 随机配置 ecc_unmask, ecc_1b_unmask, 配置寄存器 ECC_1B_SUM_CFG 为随机值 ecc_sum_num.
- 9) 循环多次产生 emu 1bit err, 若 ecc_1b_unmask 为 1, 则 emu err 传输到顶层, 反之不传输
- 10) 若 ecc_unmask 为 1, 累计 err 大于 ecc_sum_num., 校验顶层单 bit sum err 拉起。若 ecc_unmask 为 0 或累计 err 小于 ecc_sum_num., 校验顶层单 bit sum err 不拉起 (加在每个 force valid case)

2.11.14 dram_ecc_byte_err_rsp

验证场景:

- dram 测试模式下, 使能错误响应

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为测试模式、使能错误响应, 和 2bit 错误的校验码, 对 DRAM 随机的地址按照 word 写入随机数据, 再对之前写数据的地址按照 byte 方式进行读取, 校验读取 MEMECC_ADDR_2 寄存器值为刚才写入 Dram 的地址, 产生 DRAM 多 bit 错误, 不产生 1bit 错误, dbus 总线上的 hresp 为 1

2.11.15 dram_ecc_err_rsp

验证场景:

- dram 测试模式下, 使能错误响应

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为测试模式、使能错误响应, 和正确的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 且不产生 DRAM 1bit 或多 bit 错误, dbus 总线上的 hresp 为 0;
- 2) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为测试模式、使能错误响应, 和 1bit 错误的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 产生 DRAM 1bit 错误, 不产生多 bit 错误, dbus 总线上的 hresp 为 0;
- 3) 配置 SYS_CPU_ECC_CTRL2 寄存器, 使 DRAM 为测试模式、使能错误响应, 和 2bit 错误的校验码, 对 DRAM 随机的地址写入随机数据, 读出为正确数据, 产生 DRAM 多 bit 错误, 不产生 1bit 错误, dbus 总线上的 hresp 为 1

2.11.16 iram_ecc_testmode_valid

验证场景:

- iram 测试模式下, 通过寄存器配置校验码有效

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为非测试模式, 对 iram 随机的地址写入随机数据, 读出为正确数据, 且不产生 iram 1bit 或多 bit 错误, 读出 iram 的 error 地址为 0;
- 2) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为测试模式和正确的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 且不产生 iram 1bit 或多 bit 错误, 读出 iram 的 error 地址为 0;
- 3) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为测试模式和 1bit 错误的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 产生 iram 1bit 错误, 不产生多 bit 错误, 读出 iram 的 error 地址为该随机地址;
- 4) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为测试模式和 2bit 错误的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 产生 iram 多 bit 错误, 不产生 1bit 错误, 读出 iram 的 error 地址为该随机地址;
- 5) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为测试模式和 3bit 错误的校验码, 对 iram 随机的地址写入随机数据, 产生 iram 多 bit 错误或 1bit 错误其一, 读出 iram 的 error 地址为该随机地址

2.11.17 iram_ecc_invalid

验证场景:

- iram 非测试模式下, 通过寄存器配置校验码无效

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为非测试模式和正确的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 且不产生 iram 1bit 或多 bit 错误, 读出 iram 的 error 地址为 0;
- 2) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为非测试模式和 1bit 错误的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 且不产生 iram 1bit 或多 bit 错误, 读出 iram 的 error 地址为 0;
- 3) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为非测试模式和 2bit 错误的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 且不产生 iram 1bit 或多 bit 错误, 读出 iram 的 error 地址为 0;
- 4) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为非测试模式和 3bit 错误的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 且不产生 iram 1bit 或多 bit 错误, 读出 iram 的 error 地址为 0

2.11.18 iram_ecc_force_valid

验证场景:

- iram 非测试模式下, 通过 force 内部信号注入数据、校验码、地址错误

验证步骤:

- 1) 对 iram 随机的地址写入随机数据, force i_ram_rdata 为正确的数据和校验码, 读 iram 为正确数据, 且不产生 iram 1bit 或多 bit 错误, 读出 iram 的 error 地址为 0;
- 2) 对 iram 随机的地址写入随机数据, force i_ram_rdata 为 (正确的数据和校验码) 任 1bit 取反, 读 iram 为正确数据, 产生 iram 1bit 错误, 不产生多 bit 错误, 读出 iram 的 error 地址为 0;
- 3) 对 iram 随机的地址写入随机数据, force i_ram_rdata 为 (正确的数据和校验码) 任 2bit 取反, 读 iram, 产生 iram 多 bit 错误, 不产生 1bit 错误, 读出 iram 的 error 地址为该随机地址;
- 4) 对 iram 随机的地址写入随机数据, force i_ram_rdata 为 (正确的数据和校验码) 任 3bit 取反, 读 iram, 产生 iram 多 bit 错误或 1bit 错误其一, 读出 iram 的 error 地址为该随机地址;
- 5) 对 iram 随机的地址写入随机数据, force i_ram_rdata 为正确的数据和校验码, force o_ram_addr 任 1bit 取反, 读 iram 为正确数据, 产生 iram 多 bit 错误, 不产生 1bit 错误, 读出 iram 的 error 地址为注入错误后的地址;
- 6) 对 iram 随机的地址写入随机数据, force i_ram_rdata 为正确的数据和校验码, force o_ram_addr 任 2bit 取反, 读 iram 为正确数据, 产生 iram 多 bit 错误, 不产生 1bit 错误, 读出 iram 的 error 地址为注入错误后的地址;
- 7) 对 iram 随机的地址写入随机数据, force i_ram_rdata 为正确的数据和校验码, force o_ram_addr 任 3bit 取反, 读 iram 为正确数据, 产生 iram 多 bit 错误或 1bit 错误其一, 读出 iram 的 error 地址为注入错误后的地址;
- 8) 随机配置 ecc_unmask, ecc_1b_unmask, 配置寄存器 ECC_1B_SUM_CFG 为随机值 ecc_sum_num.
- 9) 循环多次产生 emu 1bit err, 若 ecc_1b_unmask 为 1, 则 emu err 传输到顶层, 否之不传输
- 10) 若 ecc_unmask 为 1, 累计 err 大于 ecc_sum_num., 校验顶层单 bit sum err 拉起。若 ecc_unmask 为 0 或累计 err 小于 ecc_sum_num., 校验顶层单 bit sum err 不拉起 (加在每个 force valid case)

2.11.19 iram_ecc_err_rsp

验证场景:

- iram 测试模式下, 使能错误响应

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为测试模式、使能错误响应, 和正确的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 且不产生 iram 1bit 或多 bit 错误, ibus 总线上的 hresp 为 0;
- 2) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为测试模式、使能错误响应, 和 1bit 错误的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 产生 iram 1bit 错误, 不产生多 bit 错误, ibus 总线上的 hresp 为 0;
- 3) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为测试模式、使能错误响应, 和 2bit 错误的校验码, 对 iram 随机的地址写入随机数据, 读出为正确数据, 产生 iram 多 bit 错误, 不产生 1bit 错误, ibus 总线上的 hresp 为 1;

2.11.20 iram_ecc_byte_err_rsp

验证场景:

- iram 测试模式下, 使能错误响应

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL1 寄存器, 使 iram 为测试模式、使能错误响应, 和 2bit 错误的校验码, 对 IRAM 随机的地址按照 word 写入随机数据, 再对之前写数据的地址按照 byte 方式进行读取, 校验读取 MEMECC_ADDR_1 寄存器值为刚才写入 ram 的地址, 产生 iram 多 bit 错误, 不产生 1bit 错误, ibus 总线上的 hresp 为 1;

2.11.21 irom_ecc_invalid

验证场景:

- irom 非测试模式下, 通过寄存器配置校验码无效

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL0 寄存器, 使 irom 为非测试模式和随机的校验码, 从 rom 随机地址读出数据, 且不产生 irom 1bit 或多 bit 错误, 读出 irom 的 error 地址为 0;
- 2) 重复步骤 1 20 次

2.11.22 irom_ecc_err_rsp

验证场景:

- irom 非测试模式下, 使能错误响应

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL0 寄存器, 使能错误响应, 从 rom 随机地址读出数据, 不产生 irom 1bit 或多 bit 错误, ibus 总线上的 hresp 为 0;
- 2) 配置 SYS_CPU_ECC_CTRL0 寄存器, 使能错误响应, 从 rom 随机地址读出数据, force rom 读出的数据有 1bit 错误, 产生 irom 1bit 错误, 不产生多 bit 错误, ibus 总线上的 hresp 为 0;
- 3) 配置 SYS_CPU_ECC_CTRL0 寄存器, 使能错误响应, 从 rom 随机地址读出数据, force rom 读出的数据有 2bit 错误, 产生 irom 多 bit 错误, 不产生 1bit 错误, ibus 总线上的 hresp 为 1;

2.11.23 irom_ecc_byte_err_rsp

验证场景:

- irom 非测试模式下, 使能错误响应

验证步骤:

- 1) 配置 SYS_CPU_ECC_CTRL0 寄存器, 使能错误响应, 从 rom 随机地址按 HALF_WORD 方式读出数据, force rom 读出的数据有 2bit 错误, 产生 irom 多 bit 错误, 不产生 1bit 错误, ibus 总线上的 hresp 为 1;

2.11.24 pke_ram0_ecc_testmode_valid

验证场景:

- pke ram0 测试模式下, 通过寄存器配置校验码有效

验证步骤:

- 1) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为非测试模式, 对 pke ram0 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram0 1bit 或多 bit 错误, 读出 pke ram0 的 error 地址为 0;
- 2) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为测试模式和正确的校验码, 对 pke ram0 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram0 1bit 或多 bit 错误, 读出 pke ram0 的 error 地址为 0;
- 3) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为测试模式和 1bit 错误的校验码, 对 pke ram0 随机的地址写入随机数据, 读出为正确数据, 产生 pke ram0 1bit 错误, 不产生多 bit 错误, 读出 pke ram0 的 error 地址为该随机地址;

- 4) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为测试模式和 2bit 错误的校验码, 对 pke ram0 随机的地址写入随机数据, 读出为正确数据, 产生 pke ram0 多 bit 错误, 不产生 1bit 错误, 读出 pke ram0 的 error 地址为该随机地址;
- 5) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为测试模式和 3bit 错误的校验码, 对 pke ram0 随机的地址写入随机数据, 产生 pke ram0 多 bit 错误或 1bit 错误其一, 读出 pke ram0 的 error 地址为该随机地址

2.11.25 pke_ram0_ecc_invalid

验证场景:

- pke ram0 非测试模式下, 通过寄存器配置校验码无效

验证步骤:

- 1) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为非测试模式和正确的校验码, 对 pke ram0 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram0 1bit 或多 bit 错误, 读出 pke ram0 的 error 地址为 0;
- 2) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为非测试模式和 1bit 错误的校验码, 对 pke ram0 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram0 1bit 或多 bit 错误, 读出 pke ram0 的 error 地址为 0;
- 3) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为非测试模式和 2bit 错误的校验码, 对 pke ram0 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram0 1bit 或多 bit 错误, 读出 pke ram0 的 error 地址为 0;
- 4) 配置 SYS_PKE_ECC_CTRL0 寄存器, 使 pke ram0 为非测试模式和 3bit 错误的校验码, 对 pke ram0 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram0 1bit 或多 bit 错误, 读出 pke ram0 的 error 地址为 0

2.11.26 pke_ram0_ecc_force_valid

验证场景:

- pke ram0 非测试模式下, 通过 force 内部信号注入数据、校验码、地址错误

验证步骤:

- 1) 对 pke ram0 随机的地址写入随机数据, force i_ram0_rdata 为正确的数据和校验码, 读 pke ram0 为正确数据, 且不产生 pke ram0 1bit 或多 bit 错误, 读出 pke ram0 的 error 地址为 0;
- 2) 对 pke ram0 随机的地址写入随机数据, force i_ram0_rdata 为 (正确的数据和校验码) 任 1bit 取反, 读 pke ram0 为正确数据, 产生 pke ram0 1bit 错误, 不产生多 bit 错误, 读出 pke ram0 的 error 地址为 0;
- 3) 对 pke ram0 随机的地址写入随机数据, force i_ram0_rdata 为 (正确的数据和校验码) 任 2bit 取反, 读 pke ram0, 产生 pke ram0 多 bit 错误, 不产生 1bit 错误, 读出 pke ram0 的 error 地址为该随机地址;

- 4) 对 pke ram0 随机的地址写入随机数据, force i_ram0_rdata 为 (正确的数据和校验码) 任 3bit 取反, 读 pke ram0, 产生 pke ram0 多 bit 错误或 1bit 错误其一, 读出 pke ram0 的 error 地址为该随机地址;
- 5) 对 pke ram0 随机的地址写入随机数据, force i_ram0_rdata 为正确的数据和校验码, force o_ram_addr 任 1bit 取反, 读 pke ram0 为正确数据, 产生 pke ram0 多 bit 错误, 不产生 1bit 错误, 读出 pke ram0 的 error 地址为注入错误后的地址;
- 6) 对 pke ram0 随机的地址写入随机数据, force i_ram0_rdata 为正确的数据和校验码, force o_ram_addr 任 2bit 取反, 读 pke ram0 为正确数据, 产生 pke ram0 多 bit 错误, 不产生 1bit 错误, 读出 pke ram0 的 error 地址为注入错误后的地址;
- 7) 对 pke ram0 随机的地址写入随机数据, force i_ram0_rdata 为正确的数据和校验码, force o_ram_addr 任 3bit 取反, 读 pke ram0 为正确数据, 产生 pke ram0 多 bit 错误或 1bit 错误其一, 读出 pke ram0 的 error 地址为注入错误后的地址;
- 8) 随机配置 ecc_unmask, ecc_1b_unmask, 配置寄存器 ECC_1B_SUM_CFG 为随机值 ecc_sum_num.
- 9) 循环多次产生 emu 1bit err, 若 ecc_1b_unmask 为 1, 则 emu err 传输到顶层, 否之不传输
- 10) 若 ecc_unmask 为 1, 累计 err 大于 ecc_sum_num., 校验顶层单 bit sum err 拉起。若 ecc_unmask 为 0 或累计 err 小于 ecc_sum_num., 校验顶层单 bit sum err 不拉起 (加在每个 force valid case)

2.11.27 pke_ram0_sec_rst_test

验证场景:

- pke ram0 错误 1bit 和 EMU 模块的对应错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 随机向 pke ram0 注入 1bit 错误;
- 2) 随机打开并检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号;
- 3) 使能非对应位的中断使能寄存器, 检查是否会触发中断, 触发则报错;
- 4) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.28 pke_ram0_ded_rst_test

验证场景:

- pke ram0 错误 2bit 和 EMU 模块的对应错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 随机向 pke ram0 注入 2bit 错误;
- 2) 随机打开并检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号;
- 3) 使能非对应位的中断使能寄存器, 检查是否会触发中断, 触发则报错;
- 4) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.29 pke_ram1_ecc_testmode_valid

验证场景:

- pke ram1 测试模式下, 通过寄存器配置校验码有效

验证步骤:

- 1) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为非测试模式, 对 pke ram1 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram1 1bit 或多 bit 错误, 读出 pke ram1 的 error 地址为 0;
- 2) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为测试模式和正确的校验码, 对 pke ram1 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram1 1bit 或多 bit 错误, 读出 pke ram1 的 error 地址为 0;
- 3) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为测试模式和 1bit 错误的校验码, 对 pke ram1 随机的地址写入随机数据, 读出为正确数据, 产生 pke ram1 1bit 错误, 不产生多 bit 错误, 读出 pke ram1 的 error 地址为该随机地址;
- 4) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为测试模式和 2bit 错误的校验码, 对 pke ram1 随机的地址写入随机数据, 读出为正确数据, 产生 pke ram1 多 bit 错误, 不产生 1bit 错误, 读出 pke ram1 的 error 地址为该随机地址;
- 5) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为测试模式和 3bit 错误的校验码, 对 pke ram1 随机的地址写入随机数据, 产生 pke ram1 多 bit 错误或 1bit 错误其一, 读出 pke ram1 的 error 地址为该随机地址;

2.11.30 pke_ram1_ecc_invalid

验证场景:

- pke ram1 非测试模式下, 通过寄存器配置校验码无效

验证步骤:

- 1) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为非测试模式和正确的校验码, 对 pke ram1 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram1 1bit 或多 bit 错误, 读出 pke ram1 的 error 地址为 0;
- 2) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为非测试模式和 1bit 错误的校验码, 对 pke ram1 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram1 1bit 或多 bit 错误, 读出 pke ram1 的 error 地址为 0;
- 3) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为非测试模式和 2bit 错误的校验码, 对 pke ram1 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram1 1bit 或多 bit 错误, 读出 pke ram1 的 error 地址为 0;
- 4) 配置 SYS_PKE_ECC_CTRL1 寄存器, 使 pke ram1 为非测试模式和 3bit 错误的校验码, 对 pke ram1 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram1 1bit 或多 bit 错误, 读出 pke ram1 的 error 地址为 0;

2.11.31 pke_ram1_ecc_force_valid

验证场景:

- pke ram1 非测试模式下, 通过 force 内部信号注入数据、校验码、地址错误

验证步骤:

- 1) 对 pke ram1 随机的地址写入随机数据, force i_ram1_rdata 为正确的数据和校验码, 读 pke ram1 为正确数据, 且不产生 pke ram1 1bit 或多 bit 错误, 读出 pke ram1 的 error 地址为 0;
- 2) 对 pke ram1 随机的地址写入随机数据, force i_ram1_rdata 为 (正确的数据和校验码) 任 1bit 取反, 读 pke ram1 为正确数据, 产生 pke ram1 1bit 错误, 不产生多 bit 错误, 读出 pke ram1 的 error 地址为 0;
- 3) 对 pke ram1 随机的地址写入随机数据, force i_ram1_rdata 为 (正确的数据和校验码) 任 2bit 取反, 读 pke ram1, 产生 pke ram1 多 bit 错误, 不产生 1bit 错误, 读出 pke ram1 的 error 地址为该随机地址;
- 4) 对 pke ram1 随机的地址写入随机数据, force i_ram1_rdata 为 (正确的数据和校验码) 任 3bit 取反, 读 pke ram1, 产生 pke ram1 多 bit 错误或 1bit 错误其一, 读出 pke ram1 的 error 地址为该随机地址;
- 5) 对 pke ram1 随机的地址写入随机数据, force i_ram1_rdata 为正确的数据和校验码, force o_ram_addr 任 1bit 取反, 读 pke ram1 为正确数据, 产生 pke ram1 多 bit 错误, 不产生 1bit 错误, 读出 pke ram1 的 error 地址为注入错误后的地址;
- 6) 对 pke ram1 随机的地址写入随机数据, force i_ram1_rdata 为正确的数据和校验码, force o_ram_addr 任 2bit 取反, 读 pke ram1 为正确数据, 产生 pke ram1 多 bit 错误, 不产生 1bit 错误, 读出 pke ram1 的 error 地址为注入错误后的地址;
- 7) 对 pke ram1 随机的地址写入随机数据, force i_ram1_rdata 为正确的数据和校验码, force o_ram_addr 任 3bit 取反, 读 pke ram1 为正确数据, 产生 pke ram1 多 bit 错误或 1bit 错误其一, 读出 pke ram1 的 error 地址为注入错误后的地址;
- 8) 随机配置 ecc_unmask, ecc_1b_unmask, 配置寄存器 ECC_1B_SUM_CFG 为随机值 ecc_sum_num.
- 9) 循环多次产生 emu 1bit err, 若 ecc_1b_unmask 为 1, 则 emu err 传输到顶层, 否之不传输
- 10) 若 ecc_unmask 为 1, 累计 err 大于 ecc_sum_num., 校验顶层单 bit sum err 拉起。若 ecc_unmask 为 0 或累计 err 小于 ecc_sum_num., 校验顶层单 bit sum err 不拉起 (加在每个 force valid case)

2.11.32 pke_ram1_sec_rst_test

验证场景:

- pke ram1 错误 1bit 和 EMU 模块的对应错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 随机向 pke ram1 注入 1bit 错误;
- 2) 随机打开并检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号;
- 3) 使能非对应位的中断使能寄存器, 检查是否会触发中断, 触发则报错;
- 4) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.33 pke_ram1_ded_rst_test

验证场景：

- pke ram1 错误 2bit 和 EMU 模块的对应错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，随机向 pke ram1 注入 2bit 错误；
- 2) 随机打开并检查对应的 EMU 硬件错误中断寄存器，ECC 错误地址以及顶层硬件错误信号；
- 3) 使能非对应位的中断使能寄存器，检查是否会触发中断，触发则报错；
- 4) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.34 pke_ram2_ecc_testmode_valid

验证场景：

- pke ram2 非测试模式下，通过 force 内部信号注入数据、校验码、地址错误

验证步骤：

- 1) 配置 SYS_PKE_ECC_CTRL2 寄存器，使 pke ram2 为非测试模式，对 pke ram2 随机的地址写入随机数据，读出为正确数据，且不产生 pke ram2 1bit 或多 bit 错误，读出 pke ram2 的 error 地址为 0；
- 2) 配置 SYS_PKE_ECC_CTRL2 寄存器，使 pke ram2 为测试模式和正确的校验码，对 pke ram2 随机的地址写入随机数据，读出为正确数据，且不产生 pke ram2 1bit 或多 bit 错误，读出 pke ram2 的 error 地址为 0；
- 3) 配置 SYS_PKE_ECC_CTRL2 寄存器，使 pke ram2 为测试模式和 2bit 错误的校验码，对 pke ram2 随机的地址写入随机数据，读出为正确数据，产生 pke ram2 1bit 错误，不产生多 bit 错误，读出 pke ram2 的 error 地址为该随机地址；
- 4) 配置 SYS_PKE_ECC_CTRL2 寄存器，使 pke ram2 为测试模式和 2bit 错误的校验码，对 pke ram2 随机的地址写入随机数据，读出为正确数据，产生 pke ram2 多 bit 错误，不产生 1bit 错误，读出 pke ram2 的 error 地址为该随机地址；
- 5) 配置 SYS_PKE_ECC_CTRL2 寄存器，使 pke ram2 为测试模式和 3bit 错误的校验码，对 pke ram2 随机的地址写入随机数据，产生 pke ram2 多 bit 错误或 1bit 错误其一，读出 pke ram2 的 error 地址为该随机地址；

2.11.35 pke_ram2_ecc_invalid

验证场景：

- pke ram2 非测试模式下，通过寄存器配置校验码无效

验证步骤:

- 1) 配置 SYS_PKE_ECC_CTRL2 寄存器, 使 pke ram2 为非测试模式和正确的校验码, 对 pke ram2 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram2 1bit 或多 bit 错误, 读出 pke ram2 的 error 地址为 0;
- 2) 配置 SYS_PKE_ECC_CTRL2 寄存器, 使 pke ram2 为非测试模式和 1bit 错误的校验码, 对 pke ram2 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram2 1bit 或多 bit 错误, 读出 pke ram2 的 error 地址为 0;
- 3) 配置 SYS_PKE_ECC_CTRL2 寄存器, 使 pke ram2 为非测试模式和 2bit 错误的校验码, 对 pke ram2 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram2 1bit 或多 bit 错误, 读出 pke ram2 的 error 地址为 0;
- 4) 配置 SYS_PKE_ECC_CTRL2 寄存器, 使 pke ram2 为非测试模式和 3bit 错误的校验码, 对 pke ram2 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram2 1bit 或多 bit 错误, 读出 pke ram2 的 error 地址为 0;

2.11.36 pke_ram2_ecc_force_valid**验证场景:**

- pke ram2 非测试模式下, 通过 force 内部信号注入数据、校验码、地址错误

验证步骤:

- 1) 对 pke ram2 随机的地址写入随机数据, force i_ram2_rdata 为正确的数据和校验码, 读 pke ram2 为正确数据, 且不产生 pke ram2 1bit 或多 bit 错误, 读出 pke ram2 的 error 地址为 0;
- 2) 对 pke ram2 随机的地址写入随机数据, force i_ram2_rdata 为 (正确的数据和校验码) 任 1bit 取反, 读 pke ram2 为正确数据, 产生 pke ram2 1bit 错误, 不产生多 bit 错误, 读出 pke ram2 的 error 地址为 0;
- 3) 对 pke ram2 随机的地址写入随机数据, force i_ram2_rdata 为 (正确的数据和校验码) 任 2bit 取反, 读 pke ram2, 产生 pke ram2 多 bit 错误, 不产生 1bit 错误, 读出 pke ram2 的 error 地址为该随机地址;
- 4) 对 pke ram2 随机的地址写入随机数据, force i_ram2_rdata 为 (正确的数据和校验码) 任 3bit 取反, 读 pke ram2, 产生 pke ram2 多 bit 错误或 1bit 错误其一, 读出 pke ram2 的 error 地址为该随机地址;
- 5) 对 pke ram2 随机的地址写入随机数据, force i_ram2_rdata 为正确的数据和校验码, force o_ram_addr 任 1bit 取反, 读 pke ram2 为正确数据, 产生 pke ram2 多 bit 错误, 不产生 1bit 错误, 读出 pke ram2 的 error 地址为注入错误后的地址;
- 6) 对 pke ram2 随机的地址写入随机数据, force i_ram2_rdata 为正确的数据和校验码, force o_ram_addr 任 2bit 取反, 读 pke ram2 为正确数据, 产生 pke ram2 多 bit 错误, 不产生 1bit 错误, 读出 pke ram2 的 error 地址为注入错误后的地址;
- 7) 对 pke ram2 随机的地址写入随机数据, force i_ram2_rdata 为正确的数据和校验码, force o_ram_addr 任 3bit 取反, 读 pke ram2 为正确数据, 产生 pke ram2 多 bit 错误或 1bit 错误其一, 读出 pke ram2 的 error 地址为注入错误后的地址;
- 8) 随机配置 ecc_unmask, ecc_1b_unmask, 配置寄存器 ECC_1B_SUM_CFG 为随机值 ecc_sum_num.

- 9) 循环多次产生 emu 1bit err, 若 ecc_1b_unmask 为 1, 则 emu err 传输到顶层, 否之不传输
- 10) 若 ecc_unmask 为 1, 累计 err 大于 ecc_sum_num., 校验顶层单 bit sum err 拉起。若 ecc_unmask 为 0 或累计 err 小于 ecc_sum_num., 校验顶层单 bit sum err 不拉起 (加在每个 force valid case)

2.11.37 pke_ram2_sec_rst_test

验证场景:

- pke ram2 错误 1bit 和 EMU 模块的对应错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 随机向 pke ram2 注入 1bit 错误;
- 2) 随机打开并检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号;
- 3) 使能非对应位的中断使能寄存器, 检查是否会触发中断, 触发则报错;
- 4) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.38 pke_ram2_ded_rst_test

验证场景:

- pke ram2 错误 2bit 和 EMU 模块的对应错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 随机向 pke ram2 注入 2bit 错误;
- 2) 随机打开并检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号;
- 3) 使能非对应位的中断使能寄存器, 检查是否会触发中断, 触发则报错;
- 4) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.39 pke_ram3_ecc_testmode_valid

验证场景:

- pke ram3 测试模式下, 通过寄存器配置校验码有效

验证步骤:

- 1) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为非测试模式, 对 pke ram3 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram3 1bit 或多 bit 错误, 读出 pke ram3 的 error 地址为 0;

- 2) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为测试模式和正确的校验码, 对 pke ram3 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram3 1bit 或多 bit 错误, 读出 pke ram3 的 error 地址为 0;
- 3) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为测试模式和 2bit 错误的校验码, 对 pke ram3 随机的地址写入随机数据, 读出为正确数据, 产生 pke ram3 1bit 错误, 不产生多 bit 错误, 读出 pke ram3 的 error 地址为该随机地址;
- 4) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为测试模式和 2bit 错误的校验码, 对 pke ram3 随机的地址写入随机数据, 读出为正确数据, 产生 pke ram3 多 bit 错误, 不产生 1bit 错误, 读出 pke ram3 的 error 地址为该随机地址;
- 5) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为测试模式和 3bit 错误的校验码, 对 pke ram3 随机的地址写入随机数据, 产生 pke ram3 多 bit 错误或 1bit 错误其一, 读出 pke ram3 的 error 地址为该随机地址;

2.11.40 pke_ram3_ecc_invalid

验证场景:

- pke ram3 非测试模式下, 通过寄存器配置校验码无效

验证步骤:

- 1) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为非测试模式和正确的校验码, 对 pke ram3 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram3 1bit 或多 bit 错误, 读出 pke ram3 的 error 地址为 0;
- 2) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为非测试模式和 1bit 错误的校验码, 对 pke ram3 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram3 1bit 或多 bit 错误, 读出 pke ram3 的 error 地址为 0;
- 3) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为非测试模式和 2bit 错误的校验码, 对 pke ram3 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram3 1bit 或多 bit 错误, 读出 pke ram3 的 error 地址为 0;
- 4) 配置 SYS_PKE_ECC_CTRL3 寄存器, 使 pke ram3 为非测试模式和 3bit 错误的校验码, 对 pke ram3 随机的地址写入随机数据, 读出为正确数据, 且不产生 pke ram3 1bit 或多 bit 错误, 读出 pke ram3 的 error 地址为 0;

2.11.41 pke_ram3_ecc_force_valid

验证场景:

- pke ram3 非测试模式下, 通过 force 内部信号注入数据、校验码、地址错误

验证步骤:

- 1) 对 pke ram3 随机的地址写入随机数据, force i_ram3_rdata 为正确的数据和校验码, 读 pke ram3 为正确数据, 且不产生 pke ram3 1bit 或多 bit 错误, 读出 pke ram3 的 error 地址为 0;

- 2) 对 pke ram3 随机的地址写入随机数据, force i_ram3_rdata 为 (正确的数据和校验码) 任 1bit 取反, 读 pke ram3 为正确数据, 产生 pke ram3 1bit 错误, 不产生多 bit 错误, 读出 pke ram3 的 error 地址为 0;
- 3) 对 pke ram3 随机的地址写入随机数据, force i_ram3_rdata 为 (正确的数据和校验码) 任 2bit 取反, 读 pke ram3, 产生 pke ram3 多 bit 错误, 不产生 1bit 错误, 读出 pke ram3 的 error 地址为该随机地址;
- 4) 对 pke ram3 随机的地址写入随机数据, force i_ram3_rdata 为 (正确的数据和校验码) 任 3bit 取反, 读 pke ram3, 产生 pke ram3 多 bit 错误或 1bit 错误其一, 读出 pke ram3 的 error 地址为该随机地址;
- 5) 对 pke ram3 随机的地址写入随机数据, force i_ram3_rdata 为正确的数据和校验码, force o_ram_addr 任 1bit 取反, 读 pke ram3 为正确数据, 产生 pke ram3 多 bit 错误, 不产生 1bit 错误, 读出 pke ram3 的 error 地址为注入错误后的地址;
- 6) 对 pke ram3 随机的地址写入随机数据, force i_ram3_rdata 为正确的数据和校验码, force o_ram_addr 任 2bit 取反, 读 pke ram3 为正确数据, 产生 pke ram3 多 bit 错误, 不产生 1bit 错误, 读出 pke ram3 的 error 地址为注入错误后的地址;
- 7) 对 pke ram3 随机的地址写入随机数据, force i_ram3_rdata 为正确的数据和校验码, force o_ram_addr 任 3bit 取反, 读 pke ram3 为正确数据, 产生 pke ram3 多 bit 错误或 1bit 错误其一, 读出 pke ram3 的 error 地址为注入错误后的地址;
- 8) 随机配置 ecc_unmask, ecc_1b_unmask, 配置寄存器 ECC_1B_SUM_CFG 为随机值 ecc_sum_num.
- 9) 循环多次产生 emu 1bit err, 若 ecc_1b_unmask 为 1, 则 emu err 传输到顶层, 否之不传输
- 10) 若 ecc_unmask 为 1, 累计 err 大于 ecc_sum_num., 校验顶层单 bit sum err 拉起。若 ecc_unmask 为 0 或累计 err 小于 ecc_sum_num., 校验顶层单 bit sum err 不拉起 (加在每个 force valid case)

2.11.42 pke_ram3_sec_rst_test

验证场景:

- pke ram3 错误 1bit 和 EMU 模块的对应错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 随机向 pke ram3 注入 1bit 错误
- 2) 随机打开并检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号
- 3) 使能非对应位的中断使能寄存器, 检查不应该上报中断
- 4) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.43 pke_ram3_ded_rst_test

验证场景:

- pke ram3 错误 2bit 和 EMU 模块的对应错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期，随机向 pke ram3 注入 1bit 错误
- 2) 随机打开并检查对应的 EMU 硬件错误中断寄存器，ECC 错误地址以及顶层硬件错误信号
- 3) 使能非对应位的中断使能寄存器，检查不应该上报中断
- 4) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.44 axi_dma_rd_err_rsp

验证场景：

- 验证 dma_axi_rd 的 soc_err 通路和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期；
- 2) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 3) 随机注入 axi 读反馈错误，通过随机串行发起 ske/hmac_axi_dma 计算，使用安全端口 key，AXI DMA 读写通道选择随机，输入数据长度随机，DMA 源/目的地址合法随机；
- 4) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 5) 检查复位寄存器随机使能对应的复位处理是否实现；
- 6) 复位后循环以上 1-5 步骤 3 次；

2.11.45 axi_dma_wr_err_rsp

验证场景：

- 验证 dma_axi_wr 的 soc_err 通路和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期；
- 2) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 3) 随机注入 axi 写反馈错误，通过随机串行发起 ske/hmac_axi_dma 计算，使用安全端口 key，AXI DMA 读写通道选择随机，输入数据长度随机，DMA 源/目的地址合法随机；
- 4) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 5) 检查复位寄存器随机使能对应的复位处理是否实现；
- 6) 复位后循环以上 1-5 步骤 3 次；

2.11.46 ahb_cfg_err_rsp_rst

验证场景：

- 验证 cfg_ahb 的 err_rsp 通路和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期；
- 2) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 3) 随机注入 hresp 反馈错误，通过 cpu 读或写一次 ahb_cfg 寄存器；
- 4) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 5) 检查复位寄存器随机使能对应的复位处理是否实现；
- 6) 复位后循环以上 1-5 步骤 3 次；

2.11.47 ahb_cfg_err_rsp_err2cpu

验证场景：

- 验证 cfg_ahb 的 err_rsp 通路和其返回给 cpu 的 err rsp

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期；
- 2) 随机使能是否返回 err rsp 给 CPU，验证 CPU 对应中断位是否拉起；
- 3) 随机注入 hresp 反馈错误，通过 cpu 读或写一次 ahb_cfg 寄存器；
- 4) 检查 cpu 接收到的 resp 信号
- 5) 复位后循环以上 1-4 步骤 3 次；

2.11.48 ahb_soc_err_rsp_rst

验证场景：

- 验证 soc_ahb 的 err_rsp 通路和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期；

- 2) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 3) 随机注入 hresp 反馈错误，通过 cpu 读或写一次 ahb_soc 寄存器；
- 4) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 5) 检查复位寄存器随机使能对应的复位处理是否实现；
- 6) 复位后循环以上 1-5 步骤 3 次；

2.11.49 ahb_soc_err_rsp_err2cpu

验证场景：

- 验证 soc_ahb 的 err_rsp 通路和其返回给 cpu 的 err rsp

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期；
- 2) 随机使能是否返回 err rsp 给 CPU，验证 CPU 对应中断位是否拉起；
- 3) 随机注入 hresp 反馈错误，通过 cpu 读或写一次 ahb_soc 寄存器；
- 4) 检查 cpu 接收到的 resp 信号
- 5) 复位后循环以上 1-4 步骤 3 次；

2.11.50 ahb_otp_err_rsp_rst

验证场景：

- 验证 otp_ahb 的 err_rsp 通路和 EMU 模块的对应的错误管理

验证步骤：

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期；
- 2) 三种错误处理的复位寄存器随机使能：复位 eHSM 寄存器，复位除了 boot 相关模块寄存器，复位 CPU 寄存器；
- 3) 随机注入 hresp 反馈错误，通过 cpu 读或写一次 ahb_otp 寄存器；
- 4) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号；
- 5) 检查复位寄存器随机使能对应的复位处理是否实现；
- 6) 复位后循环以上 1-5 步骤 3 次；

2.11.51 ahb_otp_err_rsp_before_boot

验证场景:

- 验证 otp_ahb 在 boot 启动完成前的 err_rsp 通路和 EMU 模块的对应的错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期;
- 2) 随机注入 hresp 反馈错误, 等待 cpu 复位;
- 3) 检查对应的 EMU 硬件错误中断寄存器以及顶层硬件错误信号;

2.11.52 ahb_otp_err_rsp_err2cpu

验证场景:

- 验证 otp_ahb 的 err_rsp 通路和其返回给 cpu 的 err rsp

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期;
- 2) 随机使能是否返回 err rsp 给 CPU, 验证 CPU 对应中断位是否拉起;
- 3) 随机注入 hresp 反馈错误, 通过 cpu 读或写一次 ahb_otp 寄存器;
- 4) 检查 cpu 接收到的 resp 信号
- 5) 复位后循环以上 1-4 步骤 3 次;

2.11.53 emu_wdt_timeout_test

验证场景:

- wdg 超时信号至 EMU 中断及复位

验证步骤:

- 1) 读取 ERR_HW_0 和 ERR_HW_1 复位值为 0;
- 2) 分别向 ERR_HW_0 和 ERR_HW_1 写全 1, 写无效, 读出数值为 0;
- 3) 读取 INT_HW_1 复位值为 0;
- 4) 向 INT_HW_1 写全 1, 写无效, 读出数值为 0;
- 5) 使 wdg 产生 timeout, 检查 EMU err_hw[40]=1;

- 6) (无法通过总线读取寄存器, 因 timeout 信号只保持一个 clk)
- 7) 读取中断寄存器 INT_HW_1[8]=1;
- 8) 向中断寄存器 INT_HW_1[8] 写 0, 无法清除;
- 9) 向中断寄存器 INT_HW_1 非第 8bit 写 1, 无法清除;
- 10) 向中断寄存器 INT_HW_1[8] 写 1, 能够清除;
- 11) 使 wdg 产生 timeout, CPU 顶层中断信号为 0, 遍历使能中断 INTEN_HW_1 某 1bit, 当且仅当使能第 8bit 时, CPU 顶层中断信号为 1, 其他 bit 时为 0, 向中断寄存器写 1 清除中断后, CPU 顶层中断信号为 0;
- 12) 遍历使能某 1bitCPU 复位使能 RST_CPU_HW_1, 再使 wdg 产生 timeout, 当且仅当使能第 8bit 时, EMU 输出 cpu 复位信号为 0 (只保持一个 clk), 使能其他 bit 时为 1;
- 13) 对于 all 和 noboot 复位, 与步骤 11 同理

2.11.54 emu_soc_int_test

验证场景:

- soc err 寄存器以及中断

验证步骤:

- 1) 读取 ERR_SOC_ERR,INT_SOC_ERR,INTEN_SOC_ERR 复位值为 0;
- 2) 向 ERR_SOC_ERR 写全 1, 写无效, 读出数值为 0;
- 3) 向 INT_SOC_ERR 写全 1, 写无效, 读出数值为 0;
- 4) 从 HSM 顶层给 soc_err 第 n bit 一个 2clk 的脉冲, 读取 ERR_SOC_ERR 和 INT_SOC_ERR 正确, 向 INT_SOC_ERR 写 0, 写无效; 遍历使能某 1bit 中断, 当且仅当使能与产生的中断位对应时, CPU 顶层中断信号为 1, 否则为 0;
- 5) 重复步骤 4 遍历 n=0-31
- 6) 分别给 soc_err 赋值 5a5a_5a5a,a5a5_a5a5, 全 0, 全 f, 读取 ERR_SOC_ERR 和 INT_SOC_ERR 正确

2.11.55 emu_soc_int_beforeboot_test

验证场景:

- boot 完成前验证 soc err 寄存器及中断

验证步骤:

- 1) 配置 otp, 等待硬件复位完成
- 2) 在 CPU 复位释放之前, 向 i_soc_err 注入数据, 等待 CPU 复位释放
- 3) 读取 INT_SOC_ERR 寄存器, 验证读取值与之前 i_soc_err 注入数据是否一致;

- 4) 向 ERR_SOC_ERR 写全 1, 写无效, 读出数值为 0;
- 5) 向 ERR_SOC_ERR 写全 0, 写无效, 读出数值为之前 i_soc_err 注入数据值;
- 6) 遍历使能某 1bit 中断, 当且仅当使能与产生的中断位对应时, CPU 顶层中断信号为 1, 否则为 0;

2.11.56 emu_soc_int_beforeboot_rnd_test

验证场景:

- boot 完成前验证 soc err 寄存器及中断

验证步骤:

- 1) 配置 otp, 等待硬件复位完成
- 2) 在 CPU 复位释放之前, 向 i_soc_err 注入数据, 等待 CPU 复位释放
- 3) 读取 INT_SOC_ERR 寄存器, 验证读取值与之前 i_soc_err 注入数据是否一致;
- 4) 向 ERR_SOC_ERR 写全 1, 写无效, 读出数值为 0;
- 5) 向 ERR_SOC_ERR 写全 0, 写无效, 读出数值为之前 i_soc_err 注入数据值;
- 6) 遍历使能某 1bit 中断, 当且仅当使能与产生的中断位对应时, CPU 顶层中断信号为 1, 否则为 0;
- 7) 循环随机次数;

2.11.57 emu_soc_int_duringboot_test

验证场景:

- boot 完成后验证 soc err 寄存器及中断

验证步骤:

- 1) 配置 otp, 等待硬件复位完成, 等待 boot_done 信号拉起
- 2) 在 CPU 复位释放之前, 向 i_soc_err 注入数据, 等待 CPU 复位释放
- 3) 读取 INT_SOC_ERR 寄存器, 验证读取值与之前 i_soc_err 注入数据是否一致;
- 4) 向 ERR_SOC_ERR 写全 1, 写无效, 读出数值为 0;
- 5) 向 ERR_SOC_ERR 写全 0, 写无效, 读出数值为之前 i_soc_err 注入数据值;
- 6) 遍历使能某 1bit 中断, 当且仅当使能与产生的中断位对应时, CPU 顶层中断信号为 1, 否则为 0;

2.11.58 emu_soc_int_duringboot_rnd_test

验证场景:

- boot 完成后验证 soc err 寄存器及中断

验证步骤:

- 1) 配置 otp, 等待硬件复位完成, 等待 boot_done 信号拉起
- 2) 在 CPU 复位释放之前, 向 i_soc_err 注入数据, 等待 CPU 复位释放
- 3) 读取 INT_SOC_ERR 寄存器, 验证读取值与之前 i_soc_err 注入数据是否一致;
- 4) 向 ERR_SOC_ERR 写全 1, 写无效, 读出数值为 0;
- 5) 向 ERR_SOC_ERR 写全 0, 写无效, 读出数值为之前 i_soc_err 注入数据值;
- 6) 遍历使能某 1bit 中断, 当且仅当使能与产生的中断位对应时, CPU 顶层中断信号为 1, 否则为 0;
- 7) 循环随机次数;

2.11.59 emu_soc_longtime_test**验证场景:**

- soc err 寄存器以及中断

验证步骤:

- 1) 从 HSM 顶层给 soc err 第 n bit 赋值为 1;
- 2) 重复 10 次: 读取 ERR_SOC_ERR 正确, 对 INT_SOC_ERR 第 n bit 写 1, 读取 INT_SOC_ERR 时该 bit 仍为 1;
- 3) 从 HSM 顶层给 soc err 第 n bit 赋值为 0;
- 4) 读取 ERR_SOC_ERR 正确, 对 INT_SOC_ERR 第 n bit 写 1, 读取 INT_SOC_ERR 时为 0;
- 5) 使能第 n bit 的 soc cpu 复位, 从 HSM 顶层给 soc err 第 n bit 赋值为 1, 若干时钟后赋 0, 在此期间 EMU 顶层的 cpu 复位输出信号保持为 0;
- 6) 使能第 n bit 的 nbt cpu 复位, 从 HSM 顶层给 soc err 第 n bit 赋值为 1, 若干时钟后赋 0, 在此期间 EMU 顶层的 noboot 复位输出信号保持为 0;
- 7) 重复步骤 1-6 遍历 n=0-31

2.11.60 emu_soc_cpurst_test**验证场景:**

- soc err 及其相关 cpu 复位使能

验证步骤:

- 1) 读取 RST_CPU_SOC_ERR 的复位值为 0;
- 2) 使能第 n bit 的 soc cpu 复位;
- 3) 遍历从 HSM 顶层给 soc err 某 1bit 一个 2clk 的脉冲, 当且仅当复位使能和 soc err 对应时, EMU 顶层的 cpu 复位输出信号为 0, 否则为 1, 同时 noboot 和 all 复位输出信号一直为 1
- 4) 重复步骤 2-3 遍历 n=0-31

2.11.61 emu_soc_nbtrst_test

验证场景：

- soc err 及其相关 noboot 复位使能

验证步骤：

- 1) 读取 RST_NBT_SOC_ERR 的复位值为 0；
- 2) 使能第 n bit 的 soc nbt 复位；
- 3) 遍历从 HSM 顶层给 soc err 某 1bit 一个 2clk 的脉冲，当且仅当复位使能和 soc err 对应时，EMU 顶层的 noboot 复位输出信号为 0，否则为 1，同时 cpu 和 all 复位输出信号一直为 1；
- 4) 重复步骤 2-3 遍历 n=0-31

2.11.62 emu_soc_allrst_test

验证场景：

- soc err 及其相关 all 复位使能

验证步骤：

- 1) 读取 RST_ALL_SOC_ERR 的复位值为 0；
- 2) 使能第 n bit 的 soc all 复位；
- 3) 遍历从 HSM 顶层给 soc err 某 1bit 一个 2clk 的脉冲，当且仅当复位使能和 soc err 对应时，EMU 顶层的 all 复位输出信号为 0，否则为 1，同时 cpu 和 noboot 复位输出信号一直为 1；
- 4) 重复步骤 2-3 遍历 n=0-31

2.11.63 emu_fw_test

验证场景：

- EMU FW err 寄存器可以由 SBUS 读写

验证步骤：

- 1) 1、随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE，otp 配置合法随机，等待硬件 boot_done 结束，等待 cpu 解复位；
- 2) 读取 ERR_FW_0 和 ERR_FW_1 复位值为 0；
- 3) 分别向 ERR_FW_0 和 ERR_FW_1 写入 5a5a_5a5a，读出正确；
- 4) 分别向 ERR_FW_0 和 ERR_FW_1 写入 a5a5_a5a5，读出正确；

- 5) 分别向 ERR_FW_0 和 ERR_FW_1 写入全 1, 读出正确;
- 6) 分别向 ERR_FW_0 和 ERR_FW_1 写入全 0, 读出正确;
- 7) 只向 ERR_FW_0 写入随机值, ERR_FW_0 和 ERR_FW_1 读出正确;
- 8) 只向 ERR_FW_1 写入随机值, ERR_FW_0 和 ERR_FW_1 读出正确;

2.11.64 trng_health_test_ok_in_boot

验证场景:

- 验证 trng_health_test 和 EMU 模块的对应的错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MOD 生命周期, trng reset_times 和 trng_timeout 随机, 限制跳过 skip 上电健康检测;
- 2) 不注入 trng_alarm, timeout 设置为大于等于 3;
- 3) 检查 boot 是否启动成功;
- 4) 三种错误处理的复位寄存器随机使能: 复位 eHSM 寄存器, 复位除了 boot 相关模块寄存器, 复位 CPU 寄存器;
- 5) 检查对应的 EMU 硬件错误中断寄存器, 以及顶层硬件错误信号为假;
- 6) 检查复位寄存器随机使能对应的复位处理是否实现;
- 7) 复位后循环以上 1-6 步骤 10 次

2.11.65 boot_err_test

验证场景:

- boot 注错, 顶层状态连通性验证

验证步骤:

- 1) 遍历 TEST_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE;
- 2) 通过后门对 boot 状态注错, 检查顶层 hsm_status 是否上报错误信号

2.11.66 reset_before_boot_test

验证场景:

- boot 不同状态下复位检查

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, otp 配置合法随机;
- 2) 在 boot 阶段不同状态下进行复位后, 检查是否回到 IDLE 状态, 等待 CPU 解复位成功

2.11.67 trng_health_test_fail_in_cpu

验证场景:

- 验证 trng_health_test 和 EMU 模块的对应的错误管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, reset_times, skip 上电健康检测为随机, developmode 下是否启动 trng, timeout 设置为大于等于 3;
- 2) 检查 boot 是否启动成功;
- 3) boot 启动成功后注入 trng_alarm;
- 4) 三种错误处理的复位寄存器随机使能: 复位 eHSM 寄存器, 复位除了 boot 相关模块寄存器, 复位 CPU 寄存器;
- 5) 检查对应的 EMU 硬件错误中断寄存器, 以及顶层硬件错误信号;
- 6) 检查复位寄存器随机使能对应的复位处理是否实现;
- 7) 复位后循环以上 1-6 步骤 10 次

2.11.68 emu_dram_sec_rst_test

验证场景:

- 验证 dram ecc 1bit 错误导致的 EMU 模块的对应的中断和复位管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 在 dram ecc 的测试模式下, 使 dram ecc 产生 1bit 的错误
- 2) 检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号
- 3) 使能非对应位的中断使能寄存器, 检查不应该上报中断;
- 4) 使能对应位的中断使能寄存器, 检查是否成功上报中断;
- 5) 检查复位寄存器随机使能位所对应的复位处理是否实现

2.11.69 emu_dram_ded_rst_test

验证场景:

- 验证 dram ecc 2bit 错误导致的 EMU 模块的对应的中断和复位管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 在 dram ecc 的测试模式下, 使 dram ecc 产生 2bit 的错误
- 2) 检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号
- 3) 使能非对应位的中断使能寄存器, 检查不应该上报中断;
- 4) 使能对应位的中断使能寄存器, 检查是否成功上报中断;
- 5) 检查复位寄存器随机使能位所对应的复位处理是否实现

2.11.70 emu_iram_ecc_dma_test

验证场景:

- 验证 dma 访问 iram 时产生的 ecc 错误

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, otp 配置合法随机, 等待硬件 boot_done 结束, 等待 cpu 解复位, 在 iram ecc 的测试模式下, dma 访问 iram 时, 使 iram ecc 产生 2bit 的错误;
- 2) 检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号
- 3) 使能非对应位的中断使能寄存器, 检查不应该上报中断;
- 4) 使能对应位的中断使能寄存器, 检查是否成功上报中断

2.11.71 emu_iram_sec_rst_test

验证场景:

- 验证 iram ecc 1bit 错误导致的 EMU 模块的对应的中断和复位管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 在 iram ecc 的测试模式下, 使 iram ecc 产生 1bit 的错误
- 2) 检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号
- 3) 使能非对应位的中断使能寄存器, 检查不应该上报中断;
- 4) 使能对应位的中断使能寄存器, 检查是否成功上报中断;
- 5) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.72 emu_iram_ded_rst_test

验证场景:

- 验证 iram ecc 2bit 错误导致的 EMU 模块的对应的中断和复位管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 在 iram ecc 的测试模式下, 使 iram ecc 产生 2bit 的错误
- 2) 检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号
- 3) 使能非对应位的中断使能寄存器, 检查不应该上报中断;
- 4) 使能对应位的中断使能寄存器, 检查是否成功上报中断;
- 5) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.73 emu_irom_sec_rst_test

验证场景:

- 验证 irom ecc 1bit 错误导致的 EMU 模块的对应的中断和复位管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 在 iram ecc 的非测试模式下, 使 irom ecc 产生 1bit 的错误
- 2) 检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号
- 3) 使能非对应位的中断使能寄存器, 检查不应该上报中断;
- 4) 使能对应位的中断使能寄存器, 检查是否成功上报中断;
- 5) 检查复位寄存器随机使能对应的复位处理是否实现

2.11.74 emu_irom_ded_rst_test

验证场景:

- 验证 irom ecc 2bit 错误导致的 EMU 模块的对应的中断和复位管理

验证步骤:

- 1) 随机 TEST_MODE、DEVELOP0_MODE、DEVELOP_MODE、MANUFACTURE_MODE、USER_MODE、DEBUG_MODE 生命周期, 在 irom ecc 的非测试模式下, 使 irom ecc 产生 2bit 的错误
- 2) 检查对应的 EMU 硬件错误中断寄存器, ECC 错误地址以及顶层硬件错误信号

- 3) 使能非对应位的中断使能寄存器，检查不应该上报中断；
- 4) 使能对应位的中断使能寄存器，检查是否成功上报中断；
- 5) 检查复位寄存器随机使能对应的复位处理是否实现

OSR CONFIDENTIAL